

**UNIVERSITE IBN
TOFAIL**

Faculte des Sciences

^{Kenitra}

MEMOIRE DE FIN D'ETUDES

SEWS MAROC

Sumitomo Electric

Wiring Systems

Pour l'obtention du diplome de
MASTER SCIENCES ET TECHNIQUES
Specialite : **Genie Informatique**

**Developpement d'un Systeme Andon
Intelligent**

**base sur l'Internet Industriel des
Objets**

**pour la Gestion en Temps Reel
des Pannes de Production**

Realise par :

Aissam MALKOUT

Encadre par :**Encadrant academique :**

Pr. Kaoutar ALLABOUCHE

Faculte des Sciences, Universite Ibn Tofail

Encadrant industriel :

M. Mohamed EDDOUCHE

Responsable Maintenance, SEWS Maroc

Lieu du stage :

SEWS Maroc - Departement Maintenance

Atelier Test Electrique

Zone Industrielle, Kenitra

*A mes parents,
pour leur amour inconditionnel,
leur patience et leur soutien tout au long de ce parcours.*

*A mes frères et sœurs,
pour leur encouragement constant.*

*A tous ceux qui ont contribué
de près ou de loin à la réalisation de ce travail.*

Kenitra, 2026

Remerciements

Ce travail est le fruit de la collaboration et du soutien de nombreuses personnes auxquelles je tiens à exprimer ma profonde gratitude.

Je remercie tout d'abord mon encadrant académique, **Pr. Kaoutar ALLABOUCHE**, pour sa confiance, ses conseils avisés et son accompagnement tout au long de ce projet. Ses orientations scientifiques et pédagogiques ont été déterminantes dans la réalisation de ce mémoire.

Mes remerciements s'adressent également à mon encadrant industriel, **M. Mohamed EDDOUCHE**, Responsable Maintenance à SEWS Maroc, pour m'avoir accueilli au sein de son département et pour avoir mis à ma disposition les moyens et le temps nécessaires à l'accomplissement de ce stage. Sa disponibilité et son expertise technique ont grandement enrichi ce travail.

Je remercie **M. Lahlali**, Directeur Général de SEWS Maroc, pour m'avoir permis de réaliser ce stage au sein de son entreprise, et l'ensemble du personnel du département Maintenance, en particulier les techniciens et ingénieurs de l'atelier Test Électrique, pour leur accueil chaleureux et leur collaboration.

Je tiens à remercier les membres du jury pour le temps qu'ils consacreront à l'évaluation de ce travail.

Enfin, je remercie toute ma famille et mes amis pour leur soutien moral indéfectible et leurs encouragements permanents.

Résumé

Dans le contexte de l'industrie 4.0 et de la digitalisation croissante des processus de production, la gestion efficace des pannes constitue un enjeu majeur pour les entreprises manufacturières. Ce mémoire présente la conception, le développement et la mise en oeuvre d'un système Andon intelligent basé sur l'Internet Industriel des Objets (IIoT) pour la gestion en temps réel des pannes de production chez SEWS Maroc.

Le système développé s'appuie sur une architecture matérielle composée de capteurs connectés (ESP32) interfacés avec des boutons poussoirs physiques, permettant aux opérateurs de signaler instantanément les anomalies de production. Les données sont transmises via le protocole MQTT vers un serveur backend développé en Node.js avec le framework Express, qui assure le traitement, le routage et la persistance des événements dans une base de données PostgreSQL.

Le frontend web, construit avec Angular, offre un tableau de bord interactif en temps réel permettant aux équipes de maintenance de visualiser l'état des lignes de production, de suivre la chronologie des pannes et d'analyser les métriques de performance (temps moyen de réponse, taux de disponibilité, fréquence des pannes par catégorie). Un système de notifications par e-mail et des alertes visuelles complètent la plateforme.

Les résultats expérimentaux, obtenus sur une durée de trois mois en conditions réelles de production, démontrent une amélioration significative du temps de réponse moyen aux pannes (réduction de 35%), une augmentation du taux de disponibilité des lignes (de 87% à 94%) et une meilleure traçabilité des événements. Le système a été validé sur l'atelier Test Électrique de SEWS Maroc, couvrant 8 lignes de production.

Mots-clés : Internet Industriel des Objets, Andon, MQTT, Node.js, PostgreSQL, Angular, ESP32, Gestion des pannes, Industrie 4.0

Abstract

In the context of Industry 4.0 and the growing digitalization of production processes, effective fault management represents a major challenge for manufacturing companies. This thesis presents the design, development, and implementation of an intelligent Andon system based on the Industrial Internet of Things (IIoT) for real-time production fault management at SEWS Maroc.

The developed system relies on a hardware architecture consisting of connected sensors (ESP32) interfaced with physical push buttons, enabling operators to instantly report production anomalies. Data is transmitted via the MQTT protocol to a backend server developed in Node.js with the Express framework, which handles processing, routing, and persistence of events in a PostgreSQL database.

The web frontend, built with Angular, provides an interactive real-time dashboard allowing maintenance teams to visualize production line status, track fault chronology, and analyze performance metrics (average response time, availability rate, fault frequency by category). An email notification system and visual alerts complement the platform.

Experimental results, obtained over a three-month period in real production conditions, demonstrate a significant improvement in average fault response time (35% reduction), an increase in line availability rate (from 87% to 94%), and better event traceability. The system was validated on the SEWS Maroc electrical test workshop, covering 8 production lines.

Keywords : Industrial Internet of Things, Andon, MQTT, Node.js, PostgreSQL, Angular, ESP32, Fault Management, Industry 4.0

Table des matières

Remerciements	ii
Résumé	iii
Abstract	iv
Liste des Figures	xii
Liste des Tableaux	xiii
Liste des Abréviations	xiv
1 Introduction Générale	1
1.1 Contexte général	1
1.2 Problématique	1
1.3 Objectifs du mémoire	2
1.4 Méthodologie	2
1.5 Organisation du mémoire	3
2 État de l’art	4
2.1 Le système Andon	4
2.1.1 Historique et origines	4
2.1.2 Principes de fonctionnement	4
2.1.3 Limites du système traditionnel	5
2.2 L’Internet Industriel des Objets (IIoT)	5
2.2.1 Définition et concepts fondamentaux	5
2.2.2 Architecture de référence	6
2.2.3 Avantages pour l’industrie	6
2.3 Protocoles de communication IoT	6
2.3.1 MQTT (Message Queuing Telemetry Transport)	6
2.3.2 OPC UA (Open Platform Communications Unified Architecture)	8
2.3.3 Autres protocoles	8
2.4 Technologies matérielles pour l’IIoT	8
2.4.1 Microcontrôleurs pour l’IoT	8
2.4.2 L’ESP32 : choix pour notre application	8
2.5 Technologies logicielles	9

2.5.1	Node.js et Express	9
2.5.2	PostgreSQL	9
2.5.3	Angular	10
2.6	Travaux connexes	10
2.6.1	Systèmes Andon connectés	10
2.6.2	Solutions commerciales	11
2.6.3	Positionnement de notre travail	11
2.7	Synthèse du chapitre	11
3	Contexte de l'entreprise	13
3.1	Présentation de SEWS Maroc	13
3.1.1	Le groupe SEWS	13
3.1.2	SEWS Maroc : chiffres clés	13
3.2	Organisation du département Maintenance	14
3.2.1	Structure organisationnelle	14
3.2.2	Le rôle du technicien de maintenance	14
3.3	L'atelier Test Électrique	14
3.3.1	Description de l'atelier	14
3.3.2	Processus de production	14
3.3.3	Problèmes rencontrés	15
3.4	Besoins identifiés	15
3.4.1	Besoins fonctionnels	15
3.4.2	Besoins techniques	16
3.5	Synthèse du chapitre	16
4	Analyse des besoins	17
4.1	Méthodologie d'analyse	17
4.1.1	Sources de besoins	17
4.1.2	Acteurs du système	17
4.2	Besoin fonctionnels	17
4.2.1	Module de signalisation (capteurs)	17
4.2.2	Module de traitement (backend)	18
4.2.3	Module de visualisation (frontend)	18
4.3	Besoins non fonctionnels	19
4.3.1	Performance	19
4.3.2	Fiabilité	19
4.3.3	Sécurité	19
4.3.4	Maintenabilité	19
4.4	Modélisation UML	20
4.4.1	Diagramme de cas d'utilisation	20
4.4.2	Diagramme de séquence	20
4.5	Spécifications fonctionnelles	20
4.5.1	Modèle de données préliminaire	20

4.5.2	Règles de gestion	21
4.6	Synthèse du chapitre	21
5	Conception du système	22
5.1	Architecture globale du système	22
5.1.1	Vue d'ensemble	22
5.1.2	Diagramme de déploiement	22
5.2	Architecture de la couche perception	23
5.2.1	Choix du microcontrôleur	23
5.2.2	Schéma de câblage	23
5.3	Architecture de la couche communication	24
5.3.1	Choix du protocole : MQTT	24
5.3.2	Architecture du broker MQTT	24
5.3.3	Structure des topics	24
5.4	Architecture de la couche traitement	25
5.4.1	Serveur backend : Node.js + Express	25
5.4.2	Base de données : PostgreSQL	25
5.5	Architecture de la couche présentation	25
5.5.1	Frontend web : Angular	25
5.5.2	Diagramme des composants	26
5.6	Flux de données	26
5.6.1	Flux d'un événement Andon	26
5.6.2	Flux de résolution	26
5.7	Choix d'architecture : microservices vs monolithique	27
5.7.1	Justification du choix monolithique	27
5.7.2	Perspectives d'évolution vers les microservices	27
5.8	Synthèse du chapitre	27
6	Architecture matérielle	28
6.1	Le nœud Andon ESP32	28
6.1.1	Choix du microcontrôleur	28
6.1.2	Composants du nœud	28
6.1.3	Schéma de câblage	28
6.1.4	Fonctionnement du circuit	29
6.2	Infrastructure réseau	30
6.2.1	Réseau Wi-Fi	30
6.2.2	Réseau filaire	30
6.3	Alimentation et fiabilité	30
6.3.1	Alimentation des nœuds	30
6.3.2	Robustesse industrielle	31
6.4	Dimensionnement du système	31
6.4.1	Nombre de nœuds	31
6.5	Synthèse du chapitre	31

7	Architecture logicielle	32
7.1	Organisation globale du code	32
7.2	Le serveur backend	32
7.2.1	Point d'entrée : server.js	33
7.2.2	Structure modulaire	33
7.2.3	Connexion MQTT	33
7.2.4	Diffusion Socket.IO	34
7.2.5	API REST	34
7.3	L'application frontend Angular	34
7.3.1	Modules fonctionnels	34
7.3.2	Services Angular	35
7.3.3	Routing	35
7.4	Inté-gration des composants	35
7.4.1	Flux de communication	35
7.4.2	Sécurité	36
7.5	Synthèse du chapitre	36
8	Conception de la base de données	37
8.1	Méthodologie de conception	37
8.1.1	Approche adoptée	37
8.1.2	Entités principales	37
8.2	Modèle Entité-Association	37
8.2.1	Diagramme EA	38
8.2.2	Définition des attributs	38
8.3	Modèle relationnel	38
8.3.1	Schéma relationnel	38
8.3.2	Règles d'intégrité	40
8.4	Implémentation SQL	40
8.4.1	Création des tables	40
8.4.2	Index de performance	42
8.4.3	Données de test	42
8.5	Gestion des volumes de données	42
8.5.1	Stratégie de rétention	42
8.5.2	Partitionnement temporel	43
8.6	Synthèse du chapitre	43
9	Architecture de communication	44
9.1	Protocoles de communication utilisés	44
9.1.1	MQTT : protocole principal	44
9.1.2	HTTP/HTTPS : API REST	45
9.1.3	WebSocket (Socket.IO)	45
9.2	Structure des topics MQTT	45
9.2.1	Hierarchie des topics	45

9.2.2	Format des messages	46
9.3	Flux de données détaillé	46
9.3.1	Flux d'un événement Andon	46
9.3.2	Flux de résolution	47
9.4	Gestion de la sécurité des communications	47
9.4.1	Sécurité MQTT	47
9.4.2	Sécurité HTTP	48
9.5	Gestion des erreurs et tolérance aux pannes	48
9.5.1	Stratégie de reconnexion	48
9.5.2	Persistence des données	48
9.6	Synthèse du chapitre	48
10	Implémentation du système	49
10.1	Environnement de développement	49
10.1.1	Outils utilisés	49
10.1.2	Installation des dépendances	49
10.2	Mise en place de l'infrastructure	50
10.2.1	Installation de PostgreSQL	50
10.2.2	Configuration de Mosquitto	50
10.3	Développement du firmware ESP32	50
10.3.1	Programme principal	51
10.3.2	Fonction d'envoi d'alerte	51
10.4	Développement du serveur backend	52
10.4.1	Structure du projet	52
10.4.2	Démarrage du serveur	52
10.5	Développement du frontend Angular	53
10.5.1	Structure du projet	53
10.6	Tests et validation	54
10.6.1	Tests unitaires	54
10.6.2	Tests d'intégration	54
10.7	Synthèse du chapitre	54
11	Implémentation du backend	55
11.1	Point d'entrée : server.js	55
11.1.1	Initialisation du serveur	55
11.1.2	Connexion MQTT et gestion des messages	56
11.1.3	Traitement des alertes Andon	56
11.2	Middleware d'authentification	57
11.2.1	Génération et validation des tokens JWT	57
11.3	Routes API	58
11.3.1	Route d'authentification	58
11.3.2	Routes CRUD des événements	59
11.3.3	Route de rapports	60

11.4	Gestion des notifications	61
11.4.1	Notifications par e-mail	61
11.5	Synthèse du chapitre	62
12	Implémentation du frontend	63
12.1	Initialisation du projet Angular	63
12.1.1	Création du projet	63
12.2	Composants principaux	63
12.2.1	Le tableau de bord (Dashboard)	63
12.2.2	Le template du tableau de bord	64
12.2.3	La page de détail d'une ligne	65
12.3	Services de communication	66
12.3.1	Service API REST	66
12.3.2	Service Socket.IO (temps réel)	67
12.4	Composants de visualisation	68
12.4.1	Graphiques de tendances	68
12.5	Authentification et gestion des sessions	68
12.5.1	Routeur et gardes de navigation	68
12.6	Responsive design	69
12.7	Synthèse du chapitre	69
13	Implémentation ESP32	70
13.1	Environnement de développement	70
13.1.1	Arduino IDE	70
13.1.2	Bibliothèques utilisées	70
13.2	Code source du firmware	70
13.2.1	Configuration initiale	70
13.2.2	Connexion WiFi et MQTT	71
13.2.3	Boucle principale	72
13.2.4	Envoi d'alerte Andon	73
13.2.5	Réception de commandes	74
13.3	Gestion de la fiabilité	74
13.3.1	Reconnexion automatique	74
13.3.2	Surveillance de la connexion WiFi	75
13.4	Tests du firmware	75
13.4.1	Tests unitaires	75
13.4.2	Tests en conditions réelles	76
13.5	Synthèse du chapitre	76
14	Résultats expérimentaux	77
14.1	Protocole expérimental	77
14.1.1	Conditions de test	77
14.1.2	Indicateurs mesurés	77

14.2	Résultats de performance	78
14.2.1	Temps de détection	78
14.2.2	Temps de réponse	78
14.3	Analyse des pannes	79
14.3.1	Repartition par type	79
14.3.2	Repartition par ligne	79
14.4	Indicateurs de performance	79
14.4.1	Taux de disponibilité global	79
14.4.2	Fiabilité du système	80
14.5	Retour des utilisateurs	80
14.5.1	Enquête de satisfaction	80
14.6	Coût-bénéfice	80
14.6.1	Coût du système	80
14.6.2	Bénéfices attendus	80
14.7	Synthèse du chapitre	81
15	Conclusion générale	82
15.1	Synthèse des travaux	82
15.2	Réalisation des objectifs	82
15.3	Principaux résultats	82
15.4	Difficultés rencontrées	83
15.5	Perspectives d'évolution	83
15.5.1	Court terme	83
15.5.2	Moyen terme	83
15.5.3	Long terme	84
15.6	Conclusion personnelle	84
	References Bibliographiques	85
A	Annexes	86
A.1	Annexe A : Code source du firmware ESP32	86
A.1.1	Programme principal	86
A.2	Annexe B : Script SQL	88
A.3	Annexe C : package.json	89
A.4	Annexe D : Pinout ESP32	89

Table des figures

5.1	Architecture globale du système Andon IIoT	23
5.2	Architecture du serveur backend	25
6.1	Schéma de câblage du nœud Andon	29
7.1	Organisation du code source du système	32
8.1	Diagramme Entité-Association du système Andon	38

Liste des tableaux

1	Liste des abréviations utilisées dans ce mémoire	xv
2.1	Comparaison IoTgrand public et IIoT	5
2.2	Niveaux de QoS de MQTT	7
2.3	Comparaison des microcontrôleurs pour l'IoT	9
2.4	Solutions commerciales Andon	11
3.1	Chiffres clés de SEWS Maroc	13
4.1	Acteurs du système Andon	18
6.1	Composants du nœud Andon ESP32	29
6.2	Paramètres du réseau Wi-Fi	30
6.3	Dimensionnement des nœuds	31
7.1	Endpoints de l'API REST	34
7.2	Routes de l'application Angular	36
8.1	Attributs de l'entité Ligne	39
8.2	Attributs de l'entité Evenement	39
10.1	Environnement de développement	49
13.1	Bibliothèques Arduino utilisées	70
14.1	Paramètres de l'expérimentation	77
14.2	Temps de détection par nœud	78
14.3	Comparaison des temps de reponse	78
14.4	Repartition des pannes par type	79
14.5	Pannes par ligne et taux de disponibilite	79
14.6	Indicateurs de fiabilite du systeme	80
14.7	Resultats de l'enquete de satisfaction (1-5)	80
14.8	Detail du cout du systeme	81
A.1	Pinout ESP32	89

Liste des Abréviations

Sigle	Définition
API	Application Programming Interface
CAN	Controller Area Network
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DHT	Digital Humidity Temperature
DMA	Direct Memory Access
ESP32	Microcontrôleur Wi-Fi/Bluetooth d'Espressif
GPIO	General Purpose Input/Output
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
ICMP	Internet Control Message Protocol
I ² C	Inter-Integrated Circuit
IIoT	Internet Industriel des Objets
IoT	Internet of Things
JSON	JavaScript Object Notation
LoRa	Long Range
MQTT	Message Queuing Telemetry Transport
MQTT-SN	MQTT Sensor Networks
NAT	Network Address Translation
NTP	Network Time Protocol
OLED	Organic Light-Emitting Diode
OSI	Open Systems Interconnection
OTA	Over The Air
P2P	Peer to Peer
PID	Process Identifier
QoS	Quality of Service
REST	Representational State Transfer
RGBA	Red Green Blue Alpha
RS-485	Recommended Standard 485
RTOS	Real-Time Operating System
SPI	Serial Peripheral Interface
SQL	Structured Query Language
SSR	Server-Side Rendering
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TTL	Time To Live
UART	Universal Asynchronous Receiver Transmitter
UDP	User Datagram Protocol
URI	Uniform Resource Identifier

Introduction Générale

1.1 Contexte général

L'industrie manufacturière traverse une transformation profonde sous l'impulsion de la quatrième révolution industrielle, communément dénommée Industrie 4.0. Cette révolution se caractérise par l'intégration des technologies numériques dans l'ensemble des processus de production, permettant une interconnexion sans précédent des systèmes, des machines et des hommes [1].

Dans ce contexte, la gestion des pannes de production représente un défi majeur pour les entreprises manufacturières. Les temps d'arrêt non planifiés engendrent des coûts considérables : perte de productivité, délais de livraison, dégradation de la qualité et insatisfaction client. Selon les études réalisés dans le domaine de la maintenance industrielle, les arrêts de ligne représentent en moyenne 5% à 20% du temps de production total [2].

Le système Andon, initialement développé par le constructeur automobile japonais Toyota dans le cadre du Toyota Production System (TPS), constitue un mécanisme fondamental de détection et de signalisation des anomalies en temps réel [3]. Ce système permet à tout opérateur de la chaîne de production de signaler immédiatement tout problème qu'il détecte, déclenchant ainsi une chaîne d'actions correctives rapides.

Cependant, les systèmes Andon traditionnels, basés sur des signaux visuels et sonores physiques, présentent des limites significatives dans l'environnement industriel moderne : portée limitée, absence de traçabilité numérique, difficulté d'analyse statistique et manque d'intégration avec les systèmes d'information existants.

1.2 Problématique

Face à ces limitations, la question centrale de ce mémoire est la suivante :

Comment concevoir et réaliser un système Andon intelligent, basé sur les technologies de l'Internet Industriel des Objets (IIoT), capable d'assurer la détection, la signalisation et le suivi en temps réel des pannes de production, tout en offrant des capacités d'analyse et de décision assistées ?

Cette problématique soulève plusieurs questions subsidiaires :

- Quelle architecture matérielle et logicielle est la plus adaptée pour connecter les postes de production et collecter les données en temps réel ?
- Quels protocoles de communication et quels standards technologiques permettent d'assurer la fiabilité et la performance du système ?
- Comment concevoir une base de données capable de gérer le volume et la vélocité des événements industriels ?
- Quelles métriques et quels indicateurs de performance permettent d'évaluer l'efficacité du système développé ?

1.3 Objectifs du mémoire

L'objectif principal de ce travail est la conception, le développement et la validation d'un système Andon intelligent pour SEWS Maroc. Les objectifs spécifiques sont les suivants :

1. **Analyser le contexte industriel** de SEWS Maroc et identifier les besoins fonctionnels et techniques en matière de gestion des pannes.
2. **Concevoir une architecture technique** intégrant des capteurs IoT, un serveur de traitement et une interface web en temps réel.
3. **Implémenter le système** en développant les couches matérielle (ESP32 et capteurs), logicielle (Node.js/Express, PostgreSQL) et présentation (Angular).
4. **Valider le système** en conditions réelles de production et mesurer son impact sur les indicateurs de performance.

1.4 Méthodologie

La réalisation de ce projet suit une approche structurée en plusieurs étapes :

1. **Phase d'étude et d'analyse** : immersion au sein du département Maintenance de SEWS Maroc, recueil des besoins et état de l'art des technologies IIoT.
2. **Phase de conception** : modélisation de l'architecture globale, conception de la base de données et définition des interfaces.
3. **Phase de développement** : programmation des composants matériels et logiciels, intégration des différents modules.
4. **Phase de tests et de validation** : tests unitaires, tests d'intégration, tests en conditions réelles et mesure des performances.

1.5 Organisation du mémoire

Ce mémoire est organisé en huit chapitres :

- **Chapitre 2 – État de l’art** : présente les fondements théoriques et les technologies associées au système Andon et à l’IIoT.
- **Chapitre 3 – Contexte de l’entreprise** : décrit l’environnement industriel de SEWS Maroc et les processus de production concernés.
- **Chapitre 4 – Analyse des besoins** : définit les besoins fonctionnels et non fonctionnels du système.
- **Chapitre 5 – Conception du système** : présente l’architecture globale et les choix de conception.
- **Chapitre 6 – Architecture matérielle et logicielle** : décrit les composants physiques et l’organisation logicielle.
- **Chapitre 7 – Conception de la base de données** : présente le modèle de données et l’implémentation SQL.
- **Chapitre 8 – Architecture de communication** : analyse les protocoles et flux de données du système.
- **Chapitre 9 – Implémentation** : décrit le développement concret de chaque composant.
- **Chapitre 10 – Résultats expérimentaux** : présente les tests, les mesures de performance et l’analyse des résultats.
- **Chapitre 11 – Conclusion générale** : synthèse des travaux réalisés et perspectives d’évolution.

Chapitre 2

État de l’art

Ce chapitre présente les fondements théoriques et technologiques sur lesquels repose notre travail. Nous abordons successivement le système Andon, les technologies de l’Internet Industriel des Objets, les protocoles de communication IoT, les architectures logicielles pour l’industrie, et les travaux connexes dans le domaine.

2.1 Le système Andon

2.1.1 Historique et origines

Le système Andon trouve ses origines dans le Toyota Production System (TPS), développé au Japon après la Seconde Guerre mondiale. Le mot “Andon” (jōndō en japonais) signifie littéralement “lampion” ou “lanterne” et fait référence aux signaux lumineux utilisés pour indiquer l’état d’une ligne de production [3].

Le système Andon a été conçu par Taiichi Ohno, ingénieur en chef de Toyota, comme un mécanisme de détection rapide des anomalies en production. Le principe fondamental est d’autoriser tout opérateur à arrêter la ligne de production en cas de détection d’un défaut, permettant ainsi une intervention immédiate et une correction à la source.

2.1.2 Principes de fonctionnement

Le fonctionnement d’un système Andon repose sur trois piliers fondamentaux :

1. **La détection précoce** : chaque opérateur est formé pour identifier les anomalies (défaut de qualité, panne machine, manque de matière première, situation dangereuse).
2. **La signalisation immédiate** : l’opérateur active un signal (visuel ou sonore) pour alerter l’équipe de maintenance et le responsable de ligne.
3. **La réaction rapide** : l’équipe de maintenance intervient dans les délais les plus courts pour diagnostiquer et corriger le problème.

Les signaux Andon sont généralement codés par couleur :

- **Vert** : la ligne fonctionne normalement.
- **Jaune** : anomalie détectée, intervention souhaitée.
- **Rouge** : arrêt de la ligne, intervention urgente requise.

2.1.3 Limites du système traditionnel

Bien que le système Andon traditionnel ait fait ses preuves dans de nombreux environnements industriels, il présente des limitations dans le contexte de l'industrie 4.0 :

- **Portée limitée** : les signaux visuels et sonores ne sont perceptibles que dans un rayon restreint.
- **Absence de traçabilité** : aucune donnée numérique n'est générée automatiquement.
- **Manque d'intégration** : le système fonctionne en silo, déconnecté des autres systèmes d'information.
- **Difficulté d'analyse** : pas de capacité d'analyse statistique ou de reporting automatisé.
- **Temps de réponse** : délai entre la détection et l'intervention, dû à la désinformation géographique.

2.2 L'Internet Industriel des Objets (IIoT)

2.2.1 Définition et concepts fondamentaux

L'Internet Industriel des Objets (IIoT) est l'application des technologies de l'Internet des Objets (IoT) aux environnements industriels. Il englobe l'ensemble des capteurs, actionneurs, machines et systèmes informatiques interconnectés dans un processus de production [1].

L'IIoTse distingue de l'IoTgrand public par plusieurs caractéristiques :

Critère	IoTgrand public	IIoT
Environnement	Domicile, bureau	Usine, atelier
Sensibilité aux erreurs	Tolérante	Critique
Temps réel	Non requis	Souvent requis
Sécurité	Standard	Renforcée
Volumétrie	Quelques dizaines	Des milliers
Protocoles	Wi-Fi, Bluetooth	MQTT, OPC UA, Modbus

Table 2.1. Comparaison IoTgrand public et IIoT

2.2.2 Architecture de référence

L'architecture de référence de l'IIoT généralement décrite en quatre couches :

1. **Couche perception (capteurs)** : capteurs et actionneurs qui collectent les données de l'environnement physique (température, pression, présence, etc.).
2. **Couche réseau** : infrastructure de communication qui transmet les données collectées vers les couches supérieures (Wi-Fi, Ethernet, LoRa, etc.).
3. **Couche traitement** : serveurs et plateformes qui traitent, stockent et analysent les données (cloud, edge computing).
4. **Couche application** : interfaces utilisateur, tableaux de bord et systèmes d'aide à la décision.

2.2.3 Avantages pour l'industrie

L'adoption de l'IIoT dans les environnements industriels offre de nombreux avantages :

- **Monitoring en temps réel** : surveillance continue de l'état des machines et des processus.
- **Maintenance prédictive** : anticipation des pannes grâce à l'analyse des données historiques.
- **Optimisation des processus** : identification des goulots d'étranglement et des sources de perte.
- **Traçabilité complète** : enregistrement automatique de tous les événements de production.
- **Réduction des coûts** : diminution des temps d'arrêt et des coûts de maintenance.

2.3 Protocoles de communication IoT

2.3.1 MQTT (Message Queuing Telemetry Transport)

MQTT est un protocole de messagerie léger, conçu spécifiquement pour les environnements IoT. Développé initialement par IBM en 1999, il est devenu un standard de l'OASIS (Organization for the Advancement of Structured Information Standards) [4].

Architecture publish/subscribe

MQTT repose sur un modèle publish/subscribe (pub/sub) qui découple l'émetteur du récepteur :

- **Publisher** : publie des messages sur un sujet (topic).
- **Broker** : reçoit les messages et les distribue aux abonnés.
- **Subscriber** : s'abonne à des sujets pour recevoir les messages.

Cette architecture présente des avantages pour l'IIoT :

- Dé-couplage temporel : l'émetteur et le récepteur n'ont pas besoin d'être connectés simultanément.
- Scalabilité : le broker peut gérer des milliers de connexions simultanées.
- Légèreté : les en-têtes de message sont minimaux (2 octets minimum).

Niveaux de Qualité de Service (QoS)

MQTT définit trois niveaux de Qualité de Service :

Niveau	Description
QoS 0	“At most once” : le message est envoyé une seule fois sans accusé de réception. Rapide mais sans garantie de livraison.
QoS 1	“At least once” : le message est livré au moins une fois. L'expéditeur reçoit un accusé de réception. Possibilité de doublons.
QoS 2	“Exactly once” : le message est livré exactement une fois. Le plus fiable mais le plus lent (quatre échanges).

Table 2.2. Niveaux de QoS de MQTT

Pour notre application Andon, le niveau QoS 1 constitue le meilleur compromis entre fiabilité et performance : les messages de pannes doivent être livrés de manière fiable sans la surcharge du QoS 2.

Structure des topics

Les topics MQTT suivent une hiérarchie structurée par des barres obliques (/). Pour notre système Andon, nous proposons la structure suivante :

```
1 and
2 and/<ligne>/status
3 and/<ligne>/alert
4 and/<ligne>/andon
5 and/<ligne>/capteurs
```

Listing 2.1. Structure des topics MQTT

2.3.2 OPC UA (Open Platform Communications Unified Architecture)

OPC UA est un standard de communication industrielle développé par la Fondation OPC. Contrairement à MQTT, OPC UA offre un modèle de données riche avec sémantique intégrée, ce qui le rend particulièrement adapté aux systèmes d'automatisation industrielle complexes [5].

Cependant, OPC UA présente une complexité d'implémentation plus élevée et des besoins en ressources plus importants, ce qui le rend moins adapté aux microcontrôleurs à faible consommation comme l'ESP32. Pour notre application Andon, MQTT a été préféré pour sa simplicité et sa légèreté.

2.3.3 Autres protocoles

D'autres protocoles sont utilisés dans le domaine de l'IIoT :

- **AMQP (Advanced Message Queuing Protocol)** : protocole plus lourd que MQTT, orienté entreprise.
- **CoAP (Constrained Application Protocol)** : protocole REST pour les dispositifs contraints, basé sur UDP.
- **Modbus** : protocole de communication industriel ancien, largement utilisé dans les automates programmables.
- **LoRaWAN** : protocole de réseau à longue portée et faible consommation, adapté aux zones industrielles étendues.

2.4 Technologies matérielles pour l'IIoT

2.4.1 Microcontrôleurs pour l'IoT

Le choix du microcontrôleur est un élément clé de la conception d'un système IoT industriel. Les critères principaux sont : la puissance de calcul, les capacités de communication, la consommation électrique, la fiabilité et le coût.

2.4.2 L'ESP32 : choix pour notre application

L'ESP32, développé par Espressif Systems, s'impose comme le candidat idéal pour notre système Andon en raison de ses capacités multiples :

- **Connectivité intégrée** : Wi-Fi 802.11 b/g/n et Bluetooth 4.2/BLE.
- **Puissance de calcul** : processeur dual-core à 240 MHz avec 520 KB de RAM.
- **Interface I²C** : pour la communication avec des capteurs externes.

Caractéristique	Arduino Uno	ESP8266	ESP32	STM32	Raspberry
Processeur	ATmega328P	Xtensa LX106	Xtensa LX6	ARM Cortex-M	ARM Cortex-M
Fréquence	16 MHz	80 MHz	240 MHz	168 MHz	1.4 GHz
RAM	2 KB	80 KB	520 KB	128 KB	1-8 GB
Wi-Fi	Non	Oui	Oui	Non	Oui
Bluetooth	Non	Non	Oui	Non	Non
GPIO	14	11	34	37	26
Coût	≈ 5EUR	≈ 3EUR	≈ 4EUR	≈ 10EUR	≈ 35EUR

Table 2.3. Comparaison des microcontrôleurs pour l'IoT

- **GPIO numériques** : 34 broches d'entrée/sortie, dont 16 ADC.
- **Faible coût** : prix unitaire d'environ 4 euros.
- **Communauté active** : documentation extensive et bibliothèques Arduino disponibles.

2.5 Technologies logicielles

2.5.1 Node.js et Express

Node.js est un environnement d'exécution JavaScript côté serveur, basé sur le moteur V8 de Google. Son modèle d'E/S non bloquant en fait un choix pertinent pour les applications IoT nécessitant le traitement de nombreux événements simultanés [6].

Express.js est le framework web le plus populaire pour Node.js. Il fournit une base solide pour la création d'API RESTful avec un minimum de surcharge :

- **Performance** : le modèle event-driven permet de gérer des milliers de connexions simultanées.
- **Simplicité** : syntaxe intuitive et large écosystème de middleware.
- **Socket.IO** : inté-gration native pour les communications temps réel bidirectionnelles.
- **npm** : gestionnaire de paquets avec plus d'un million de bibliothèques.

2.5.2 PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle open source, reconnu pour sa fiabilité, ses capacités d'extension et sa conformité aux standards SQL [7].

Pour notre application, PostgreSQL offre des avantages spécifiques :

- **Type JSON** : support natif du type JSONB pour stocker les données de capteurs de forme variable.
- **Partitionnement** : capacité de partitionnement des tables pour gérer les volumes importants d'événements.
- **Indexation** : indexation avancée pour les requêtes temporelles.
- **Extensions** : possibilité d'ajouter des extensions comme PostGIS ou TimescaleDB pour l'analyse temporelle.
- **ACID** : garantie des propriétés Transactionnelles même en cas de défaut.

2.5.3 Angular

Angular est un framework de développement web développé par Google, utilisant le langage TypeScript. Sa structure modulaire et ses outils de développement en font un choix approprié pour les applications industrielles nécessitant des interfaces dynamiques et réactives.

Les fonctionnalités clés d'Angular pour notre application incluent :

- **Composants** : architecture basée sur des composants réutilisables.
- **Binding bidirectionnel** : synchronisation automatique entre le modèle de données et la vue.
- **RxJS** : bibliothèque de programmation réactive pour la gestion des flux de données en temps réel.
- **Services** : injection de dépendance et services pour la communication avec le backend.
- **TypeScript** : typage statique pour une meilleure maintenabilité du code.

2.6 Travaux connexes

2.6.1 Systèmes Andon connectés

Plusieurs travaux de recherche ont exploré l'intégration des technologies IoT dans les systèmes Andon :

- **Wang et al. (2019)** ont proposé un système Andon basé sur des tags RFID pour le suivi en temps réel des événements de production. Leur approche utilisait des lecteurs RFID fixes le long de la ligne de production pour détecter automatiquement les mouvements des pièces et déclencher des alertes en cas d'anomalie.

- **Zhang et al. (2020)** ont développé une plateforme Andon basé sur le cloud avec des capteurs sans fil. Leur système inté-grait un module d’analyse prédictive utilisant l’apprentissage automatique pour anticiper les pannes.
- **Liu et al. (2021)** ont présenté un système Andon intelligent utilisant des camé-ras et le traitement d’image pour la détection automatique des défauts visuels.

2.6.2 Solutions commerciales

Plusieurs solutions commerciales existent sur le marché :

Solution	Fournisseur	Caractéristiques
Andon Board	Asprova	Tableau Andon numérique
ThingWorx	PTC	Plateforme IIoT complète
Ignition	Inductive Automation	SCADA + Andon
SFA Andon	SFA	Système Andon modulaire

Table 2.4. Solutions commerciales Andon

2.6.3 Positionnement de notre travail

Notre travail se distingue des approches existantes par plusieurs aspects :

1. **Approche complète** : nous couvrons l’intégralité de la chaîne, des capteurs physiques à l’interface utilisateur.
2. **Technologies open source** : utilisation exclusive de technologies open source (ESP32, Node.js, PostgreSQL, Angular).
3. **Adaptation au contexte** : le système est spécifiquement conçu pour les contraintes de l’atelier Test Électrique de SEWS Maroc.
4. **Coût maîtrisé** : le coût total du système est significativement inférieur aux solutions commerciales.

2.7 Synthèse du chapitre

Ce chapitre a permis de poser les fondements théoriques et technologiques de notre travail. Le système Andon, né dans le cadre du TPS, reste un outil fondamental de gestion des pannes en production. Son inté-gration avec les technologies IIoT, et en particulier avec le protocole MQTT et les microcontrôleurs connectés comme l’ESP32, permet de dé-passer ses limitations historiques. Les technologies logicielles choisies (Node.js, PostgreSQL, Angular) forment un ensemble cohérent et performant pour la

réalisation de notre système Andon intelligent.

Contexte de l'entreprise

3.1 Présentation de SEWS Maroc

3.1.1 Le groupe SEWS

Sumitomo Electric Wiring Systems (SEWS) est une filiale du groupe japonais Sumitomo Electric Industries, l'un des plus grands fabricants mondiaux de câbles et harnais automobiles. Fondée en 1917, le groupe Sumitomo Electric emploie plus de 240 000 collaborateurs dans le monde et présente un chiffre d'affaires annuel d'environ 30 milliards de dollars [8].

SEWS Maroc a été créé en 2000 à Kénitra, dans la zone industrielle, dans le cadre de la stratégie d'expansion internationale du groupe. L'entreprise spécialise dans la fabrication de harnais

'electriques pour l'industrie automobile, servant des constructeurs tels que Renault, PSA (Stellantis) et d'autres constructeurs internationaux.

3.1.2 SEWS Maroc : chiffres clés

Indicateur	Valeur
Date de création	2000
Effectif	1 200 employés
Chiffre d'affaires	≈ 150 millions EUR
Surface d'exploitation	20 000 m ²
Lignes de production	15
Capacité annuelle	3 millions de harnais
Principaux clients	Renault, Stellantis, Toyota

Table 3.1. Chiffres clés de SEWS Maroc

3.2 Organisation du département Maintenance

3.2.1 Structure organisationnelle

Le département Maintenance de SEWS Maroc est organisé sous la responsabilité de M. Mohamed EDDOUCH. Il comprend les équipes suivantes :

- **Maintenance électrique** : diagnostic et réparation des défauts électriques sur les harnais.
- **Maintenance mécanique** : entretien et réparation des machines et outillages.
- **Maintenance informatique** : support des systèmes d'information et des réseaux.
- **Métrologie** : calibration et vérification des instruments de mesure.

3.2.2 Le rôle du technicien de maintenance

Le technicien de maintenance joue un rôle central dans le bon fonctionnement des lignes de production. Ses missions principales sont :

1. **Diagnostic** : identifier l'origine de la panne en analysant les symptômes.
2. **Intervention** : réaliser les réparations nécessaires dans les délais imposés.
3. **Prévention** : réaliser les opérations d'entretien préventif.
4. **Documentation** : enregistrer les interventions et les pièces utilisées.
5. **Amélioration** : proposer des actions correctives et préventives.

3.3 L'atelier Test Électrique

3.3.1 Description de l'atelier

L'atelier Test Électrique est l'un des ateliers critiques de SEWS Maroc. Il est dédié à la vérification

électrique des harnais produits, garantissant leur conformité aux spécifications des clients. Cet atelier compte 8 lignes de production, chacune équipée de machines de test automatisées.

3.3.2 Processus de production

Le processus de production dans l'atelier Test Électrique suit les étapes suivantes :

1. **Réception** : les harnais semi-finis sont réceptionnés de l'atelier précédent.

2. **Préparation** : les harnais sont positionnés sur les gabarits de test.
3. **Test**
 'electrique : la machine réalise les mesures de résistance, de court-circuit et d'ouverture.
4. **Validation** : les résultats sont comparés aux spécifications du client.
5. **Conditionnement** : les harnais validés sont conditionnés pour l'expédition.

3.3.3 Problèmes rencontrés

L'analyse du fonctionnement actuel de l'atelier a révélé plusieurs problèmes :

- **Signalisation manuelle** : les pannes sont signalées oralement ou par radio, sans traçabilité numérique.
- **Temps de réponse variable** : le délai entre la détection et l'intervention varie de 5 à 30 minutes.
- **Absence de statistiques** : aucun outil de suivi des pannes et de leurs durées n'est disponible.
- **Perte d'informations** : les données de production sont partiellement enregistrées manuellement.
- **Difficulté d'analyse** : l'identification des tendances et des causes racines est laborieuse.

3.4 Besoins identifiés

3.4.1 Besoins fonctionnels

L'immersion au sein de l'atelier Test Électrique a permis d'identifier les besoins fonctionnels suivants :

1. **Signalisation numérique** : remplacement des signaux physiques par des alertes numériques.
2. **Géolocalisation des pannes** : identification automatique de la ligne et du poste touché.
3. **Suivi en temps réel** : affichage en temps réel de l'état de chaque ligne.
4. **Historique** : archivage de tous les événements pour analyse ultérieure.
5. **Notifications** : envoi automatique d'alertes aux équipes concernées.
6. **Rapports** : génération automatique de rapports et statistiques.

3.4.2 Besoins techniques

1. **Fiabilité** : le système doit fonctionner 24h/24, 7j/7, sans interruption.
2. **Temps réel** : les données doivent être affichées en moins de 2 secondes.
3. **Compatibilité** : le système doit s'intégrer avec l'infrastructure réseau existante.
4. **Sécurité** : les accès doivent être contrôlés et les données protégées.
5. **Extensibilité** : le système doit pouvoir être étendu à d'autres ateliers.
6. **Coût** : le budget alloué est limité et le système doit être économique.

3.5 Synthèse du chapitre

Ce chapitre a permis de décrire l'environnement industriel dans lequel s'inscrit notre travail. SEWS Maroc, entreprise spécialisée dans la fabrication de harnais automobiles, dispose d'un atelier Test Électrique où les pannes de production sont gérées de manière manuelle et sans traçabilité numérique. L'analyse des besoins a mis en évidence la nécessité d'un système Andon numérique et connecté pour améliorer la réactivité des équipes de maintenance et optimiser la gestion des pannes.

Analyse des besoins

Ce chapitre présente l'analyse détaillée des besoins fonctionnels et non fonctionnels du système Andon à développer. L'approche adoptée combine le recueil des besoins par observation directe et entretiens, et leur formalisation à l'aide du langage de modélisation UML.

4.1 Méthodologie d'analyse

4.1.1 Sources de besoins

Les besoins ont été collectés à partir de plusieurs sources :

- **Observations directes** : immersion de trois mois au sein de l'atelier Test Électrique.
- **Entretiens** : conversations avec les techniciens de maintenance, les chefs d'atelier et le responsable maintenance.
- **Documentation existante** : fiches d'intervention, rapports d'arrêt, tableaux de suivi.
- **État de l'art** : analyse des solutions existantes dans la littérature.

4.1.2 Acteurs du système

L'identification des acteurs est une étape fondamentale de l'analyse. Le système Andon prévu intègre les acteurs suivants :

4.2 Besoin fonctionnels

4.2.1 Module de signalisation (capteurs)

Ce module correspond à la couche physique de collecte des événements :

- **F-SIG-01** : Détection de l'appui sur le bouton Andon physique.

Acteur	Rôle
Opérateur de production	Signale les anomalies en appuyant sur le bouton Andon de son poste de travail.
Technicien de maintenance	Intervient pour diagnostiquer et ré-soudre les pannes signalées.
Chef d'atelier	Supervise l'ensemble des lignes et gère les priorités d'intervention.
Responsable maintenance	Analyse les statistiques et optimise les processus de maintenance.
Administrateur système	Configure et administre le système Andon.

Table 4.1. Acteurs du système Andon

- **F-SIG-02** : Identification automatique de la ligne de production.
- **F-SIG-03** : Envoi immédiat de la notification au serveur.
- **F-SIG-04** : Indication visuelle locale (LED verte/rouge) de l'état.
- **F-SIG-05** : Résistance aux conditions industrielles (poussière, vibrations, température).

4.2.2 Module de traitement (backend)

Ce module assure le traitement centralisé des événements :

- **F-BACK-01** : Réception et validation des messages MQTT.
- **F-BACK-02** : Enregistrement des événements dans la base de données.
- **F-BACK-03** : Gestion des utilisateurs et des droits d'accès.
- **F-BACK-04** : Génération des notifications (e-mail, alertes visuelles).
- **F-BACK-05** : Exposition d'une API REST pour le frontend.
- **F-BACK-06** : Diffusion des événements en temps réel via WebSocket.
- **F-BACK-07** : Calcul des métriques et indicateurs de performance.

4.2.3 Module de visualisation (frontend)

Ce module fournit l'interface utilisateur :

- **F-FRONT-01** : Tableau de bord principal avec vue d'ensemble des lignes.
- **F-FRONT-02** : Détail de chaque ligne avec chronologie des événements.
- **F-FRONT-03** : Vue temps réel des alertes actives.
- **F-FRONT-04** : Rapports et statistiques (par période, par ligne, par type).

- **F-FRONT-05** : Authentification et gestion des profils utilisateurs.
- **F-FRONT-06** : Responsive design pour utilisation sur tablette et smart-phone.
- **F-FRONT-07** : Historique des pannes avec capacité de recherche et filtrage.

4.3 Besoins non fonctionnels

4.3.1 Performance

- **NF-PERF-01** : Le temps de transmission d'un événement du capteur au serveur doit être inférieur à 1 seconde.
- **NF-PERF-02** : Le frontend doit afficher les données en temps réel avec une latence inférieure à 2 secondes.
- **NF-PERF-03** : Le serveur doit supporter au moins 50 connexions simultanées.
- **NF-PERF-04** : Les requêtes API doivent répondre en moins de 500 ms.

4.3.2 Fiabilité

- **NF-FIAB-01** : Le système doit être disponible 99,5% du temps.
- **NF-FIAB-02** : Les données ne doivent jamais être perdues (persistance fiable).
- **NF-FIAB-03** : Le système doit gérer les pannes de réseau sans perte de données.

4.3.3 Sécurité

- **NF-SEC-01** : Authentification requise pour tout accès au système.
- **NF-SEC-02** : Les mots de passe doivent être hashés et salés.
- **NF-SEC-03** : Les données doivent être transmises de manière sécurisée (TLS/HTTPS).
- **NF-SEC-04** : Journalisation de toutes les actions critiques.

4.3.4 Maintenabilité

- **NF-MAINT-01** : Le code doit suivre les bonnes pratiques de développement.
- **NF-MAINT-02** : La documentation technique doit être complète et à jour.
- **NF-MAINT-03** : Le système doit être modulaire pour faciliter les évolutions.

4.4 Modélisation UML

4.4.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation décrit les interactions entre les acteurs et le système. Les cas d'utilisation principaux sont :

1. **Signaler une panne** : l'opérateur appuie sur le bouton Andon.
2. **Consulter le tableau de bord** : le chef d'atelier visualise l'état des lignes.
3. **Diagnostiquer une panne** : le technicien accède aux détails de la panne.
4. **Clôturer une intervention** : le technicien enregistre la résolution.
5. **Consulter les rapports** : le responsable maintenance analyse les statistiques.
6. **Gérer les utilisateurs** : l'administrateur configure les accès.

4.4.2 Diagramme de séquence

Le scénario principal de signalement d'une panne suit le flux suivant :

1. L'opérateur appuie sur le bouton Andon physique.
2. L'ESP32 détecte l'appui et génère un message JSON.
3. Le message est publié sur le topic MQTT correspondant.
4. Le serveur Node.js reçoit le message via la connexion MQTT.
5. L'événement est enregistré dans PostgreSQL.
6. Le serveur diffuse l'événement via Socket.IO.
7. Le frontend Angular reçoit la notification et met à jour l'affichage.
8. Un e-mail de notification est envoyé au technicien de garde.

4.5 Spécifications fonctionnelles

4.5.1 Modèle de données préliminaire

Les entités principales du système sont définies comme suit :

- **Ligne** : identifiant, nom, numéro, statut (active/inactive/en panne).
- **Evenement** : identifiant, date, type, statut, id_ligne, description.
- **Utilisateur** : identifiant, nom, prénom, e-mail, rôle, mot_de_passe.
- **Notification** : identifiant, date, type, destinataire, contenu, statut.

4.5.2 Règles de gestion

1. Un même bouton Andon ne peut générer qu'un seul événement actif à la fois.
2. La clôture d'un événement requiert la saisie d'un compte-rendu d'intervention.
3. Seul un administrateur peut créer ou supprimer des comptes utilisateurs.
4. Les rapports ne peuvent être générés que pour des périodes inférieures à 12 mois.
5. Toute action critique (clôture, suppression) doit être tracée dans le journal.

4.6 Synthèse du chapitre

L'analyse des besoins a permis de formaliser les exigences fonctionnelles et non fonctionnelles du système Andon. Quatre modules principaux ont été identifiés : signalisation (capteurs), traitement (backend), visualisation (frontend) et notifications. Les contraintes de performance, fiabilité et sécurité ont été définies pour guider la phase de conception détaillée.

Conception du système

Ce chapitre présente la conception globale du système Andon IIoT, en définissant l'architecture technique, les choix technologiques et les mécanismes d'intégration entre les différents composants.

5.1 Architecture globale du système

5.1.1 Vue d'ensemble

Le système Andon développé suit une architecture en couches, permettant une séparation claire des responsabilités et facilitant la maintenance et l'évolution de chaque composant.

L'architecture se compose de quatre couches principales :

1. **Couche perception** : dispositifs physiques de détection et de signalisation.
2. **Couche communication** : protocoles et infrastructure réseau.
3. **Couche traitement** : serveur backend et base de données.
4. **Couche présentation** : interface web et notifications.

5.1.2 Diagramme de déploiement

Le diagramme de déploiement illustre la répartition physique des composants du système :

- **Nœud ESP32 (x8)** : un par ligne de production, connecté au réseau Wi-Fi de l'atelier.
- **Serveur backend** : machine virtuelle ou physique hébergeant Node.js, PostgreSQL et le broker MQTT.
- **Postes clients** : navigateurs web sur PC, tablettes ou smartphones pour accéder au frontend Angular.

Architecture globale du système Andon IIoT

(Figure a insérer)

Figure 5.1. Architecture globale du système Andon IIoT

5.2 Architecture de la couche perception

5.2.1 Choix du microcontrôleur

Après analyse comparative (cf. Table 2.3), l'ESP32 a été sélectionné pour les raisons suivantes :

- Connectivité Wi-Fi intégrée, éliminant le besoin de modules externes.
- Processeur dual-core à 240 MHz, suffisant pour le traitement local.
- Faible consommation (< 240 mA en transmission Wi-Fi).
- Prix unitaire compétitif (≈ 4 EUR).
- Communauté Arduino active, facilitant le développement.

5.2.2 Schéma de câblage

Le circuit de chaque nœud Andon comprend :

- Un bouton poussoir rouge (activation Andon).
- Une LED verte (ligne OK).
- Une LED rouge (panne active).
- Un connecteur d'alimentation 5V.
- Une résistance de pull-down ($10\text{ k}\Omega$) pour le bouton.

La broche GPIO4 est utilisée pour la détection de l'appui sur le bouton, et les broches GPIO16 et GPIO17 contrôlent les LED verte et rouge respectivement.

5.3 Architecture de la couche communication

5.3.1 Choix du protocole : MQTT

Le protocole MQTT a été retenu pour sa pertinence dans le contexte IoT :

- Légèreté des messages (en-têtes minimaux).
- Le modèle publish/subscribe, idéal pour la dé-couplage.
- Les niveaux de QoS, garantissant la livraison des messages critiques.
- La prise en charge native par de nombreuses bibliothèques IoT.

5.3.2 Architecture du broker MQTT

Le broker MQTT constitue le cœur du réseau de communication. Nous avons choisi **Mosquitto**, un broker open source léger et fiable :

- Installation locale sur le serveur backend.
- Port 1883 pour les connexions standard (TCP).
- Port 8883 pour les connexions sécurisées (TLS).
- Authentification par identifiant/mot de passe.
- Persistance des messages pour les abonnés déconnectés.

5.3.3 Structure des topics

La hiérarchie des topics MQTT est conçue pour refléter l'organisation physique de l'atelier :

```
1 # Topic principal pour les alertes Andon
2 sews/andon/<ligne_id>/alert
3
4 # Topic pour l'\`etat des capteurs
5 sews/capteurs/<ligne_id>/status
6
7 # Topic pour les mesures de temp\`erature
8 sews/capteurs/<ligne_id>/temperature
9
10 # Topic pour les commandes retour
11 sews/command/<ligne_id>/led
```

Listing 5.1. Hiérarchie des topics MQTT

5.4 Architecture de la couche traitement

5.4.1 Serveur backend : Node.js + Express

Le serveur backend est développé en Node.js avec le framework Express, offrant :

- Un serveur HTTP pour l'API REST.
- Une connexion MQTT au broker Mosquitto.
- Un serveur WebSocket (Socket.IO) pour les mises à jour en temps réel.
- La logique métier de traitement des événements.
- L'interface avec PostgreSQL pour la persistance des données.

Architecture du serveur backend

(Figure à insérer)

Figure 5.2. Architecture du serveur backend

5.4.2 Base de données : PostgreSQL

PostgreSQL est utilisé pour la persistance des données :

- Stockage des événements Andon (création, modification, clôture).
- Gestion des utilisateurs et des droits d'accès.
- Historique des notifications envoyées.
- Données agrégées pour le calcul des indicateurs.

5.5 Architecture de la couche présentation

5.5.1 Frontend web : Angular

L'interface utilisateur est développée en Angular, offrant :

- Un tableau de bord interactif avec vue d'ensemble des lignes.
- Des graphiques de tendances et d'analyses.
- La mise à jour en temps réel grâce à Socket.IO.
- La responsive design pour l'utilisation sur différents terminaux.
- L'authentification sécurisée avec gestion des sessions.

5.5.2 Diagramme des composants

Les composants principaux du frontend sont organisés en modules :

- **Module Dashboard** : composant principal avec vue d'ensemble.
- **Module Lignes** : liste et détail des lignes de production.
- **Module Evenements** : historique et gestion des pannes.
- **Module Rapports** : génération et consultation des statistiques.
- **Module Admin** : gestion des utilisateurs et configuration.

5.6 Flux de données

5.6.1 Flux d'un événement Andon

Le parcours complet d'un événement Andon suit les étapes suivantes :

1. L'opérateur appuie sur le bouton physique.
2. L'ESP32 génère un message JSON avec : id_ligne, timestamp, type.
3. Le message est publié sur le topic MQTT `sews/andon/{ligne_id}/alert`.
4. Le broker MQTT redistribue le message aux abonnés.
5. Le serveur Node.js reçoit et traite le message.
6. L'événement est enregistré dans PostgreSQL.
7. Le serveur diffuse l'événement via Socket.IO.
8. Le frontend Angular reçoit la notification et met à jour l'affichage.
9. Une notification e-mail est envoyée si nécessaire.
10. La LED rouge de l'ESP32 s'allume pour signaler visuellement la panne.

5.6.2 Flux de résolution

Lorsqu'un technicien réésout une panne :

1. Le technicien accède à l'événement dans le frontend.

2. Il saisit le compte-rendu d'intervention.
3. Le frontend envoie une requête PUT à l'API REST.
4. Le serveur met à jour PostgreSQL et publie la clôture sur MQTT.
5. L'ESP32 reçoit la commande et éteint la LED rouge (allume le vert).
6. Le frontend affiche la ligne comme ré-solue.

5.7 Choix d'architecture : microservices vs monolithique

5.7.1 Justification du choix monolithique

Pour un projet de cette envergure, une architecture monolithique a été privilégiée pour les raisons suivantes :

- **Simplicité** : un seul processus à déployer et à maintenir.
- **Performance** : pas de surcharge de communication inter-services.
- **Cohérence** : transactions garanties au sein d'un même processus.
- **Coût** : infrastructure réduite, adaptée au budget du projet.
- **Échelle** : le système gère 8 lignes, ce qui ne nécessite pas de distribution.

5.7.2 Perspectives d'évolution vers les microservices

Si le système devait être étendu à plusieurs sites de production, une migration vers une architecture microservices pourrait être envisagée, en séparant notamment :

- Un service MQTT dé-dédié à la réception des messages.
- Un service de traitement et d'agrégation.
- Un service de notification.
- Un service d'API pour le frontend.

5.8 Synthèse du chapitre

La conception du système Andon IIoT repose sur une architecture en quatre couches : perception (ESP32), communication (MQTT), traitement (Node.js/PostgreSQL) et présentation (Angular). Le choix d'une architecture monolithique est justifié par l'envergure du projet et les contraintes de développement. La structure des topics MQTT et le flux de données ont été conçus pour assurer la fiabilité et la réactivité du système en conditions industrielles.

Architecture matérielle

Ce chapitre décrit en détail les composants matériels du système Andon, depuis les nœuds de détection jusqu'à l'infrastructure réseau.

6.1 Le nœud Andon ESP32

6.1.1 Choix du microcontrôleur

L'ESP32 (Espressif Systems) a été sélectionné comme microcontrôleur principal pour chaque nœud Andon. Ce choix est motivé par :

- Connectivité Wi-Fi 802.11 b/g/n et Bluetooth 4.2/BLE intégrées.
- Processeur dual-core Xtensa LX6 à 240 MHz avec 520 KB de RAM.
- 34 broches GPIO dont 16 ADC (convertisseur analogique-numérique).
- Interface I²C et SPI pour l'ajout de capteurs externes.
- Consommation réduite : 80 mA en mode Wi-Fi actif.
- Prix unitaire compétitif ($\approx$ 4EUR).
- Soutien Arduino et MicroPython, facilitant le développement.

6.1.2 Composants du nœud

Chaque nœud Andon est composé des éléments suivants :

6.1.3 Schéma de câblage

Le schéma de câblage définit les connexions électriques du nœud Andon :

- Le bouton poussoir est connecté entre la broche **GPIO4** et la masse (GND), avec une résistance de pull-down de 10 k Ω pour garantir un état logique bas défini au repos.
- La **LED verte** est connecté entre la broche **GPIO16** et la masse, série d'une résistance de 220 Ω .

Composant	Référence	Fonction
Microcontrôleur	ESP32-DevKitC	Processeur, Wi-Fi, BT
Bouton poussoir	KY-004	Signalisation Andon
LED verte	LED 5mm 3V	Indicateur ligne OK
LED rouge	LED 5mm 3V	Indicateur panne
Résistances	220 Ω (x2)	Limitation courant LED
Résistance pull-down	10 k Ω	Stabilisation signal bouton
Alimentation	USB 5V / adaptateur	Alimentation du nœud
Boîtier	Boîtier plastique IP54	Protection du circuit

Table 6.1. Composants du nœud Andon ESP32

- La **LED rouge** est connecté entre la broche **GPIO17** et la masse, série d'une résistance de 220 Ω .
- L'alimentation est fournie via le connecteur USB du module ESP32 (5V).

Schéma de câblage du nœud Andon

(Figure à insérer)

Figure 6.1. Schéma de câblage du nœud Andon

6.1.4 Fonctionnement du circuit

Le circuit fonctionne selon le principe suivant :

1. Au repos, la broche GPIO4 est à l'état bas (0V) grâce à la résistance pull-down.
2. Lorsque l'opérateur appuie sur le bouton, le signal passe à l'état haut (3,3V).
3. L'ESP32 détecte le front montant par interruption logicielle.
4. Après débounce (200 ms), l'ESP32 génère un message JSON.
5. Le message est publié sur le topic MQTT correspondant.

6. L'ESP32 allume la LED rouge en attendant la confirmation du serveur.

6.2 Infrastructure réseau

6.2.1 Réseau Wi-Fi

Le réseau Wi-Fi de l'atelier assure la communication entre les nœuds ESP32 et le broker MQTT :

Paramètre	Valeur
Standard	Wi-Fi 802.11 b/g/n (2,4 GHz)
Protocole de sécurité	WPA2-Enterprise
Port dédié aux IoT	VLAN séparé
Filtrage	Par adresse MAC
Bande passante	1 Mb/s dédié par nœud
Portée	30–50 m (atelier)

Table 6.2. Paramètres du réseau Wi-Fi

6.2.2 Réseau filaire

Le serveur backend est connecté au réseau filaire de l'entreprise :

- Connectivité Ethernet Gigabit.
- Adresse IP statique dans le sous-réseau dédié.
- Pare-feu configuré pour limiter les accès aux ports nécessaires (1883 MQTT, 3000 API, 5432 PostgreSQL).
- Pas de connexion Internet directe pour des raisons de sécurité.

6.3 Alimentation et fiabilité

6.3.1 Alimentation des nœuds

Chaque nœud Andon est alimenté par :

- Un adaptateur secteur 5V/2A branché sur une prise à proximité du poste.
- En secours : un module de backup batteries (autonomie 4h).

6.3.2 Robustesse industrielle

Les nœuds doivent résister aux conditions de l'atelier :

- Température ambiante : 15°C à 35°C.
- Humidité relative : 30% à 80%.
- Poussière : boîtier IP54 recommandé.
- Vibrations : fixation solide du boîtier au poste de travail.
- Perturbations électromagnétiques : blindage du boîtier métallique.

6.4 Dimensionnement du système

6.4.1 Nombre de nœuds

Le système est dimensionné pour l'atelier Test Électrique de SEWS Maroc :

Paramètre	Valeur
Nombre de lignes	8
Nœuds par ligne	1
Nœuds total	8
Boutons par ligne	1
LEDs par ligne	2 (rouge + verte)
Coût par nœud	≈ 12EUR
Coût total capteurs	≈ 96EUR

Table 6.3. Dimensionnement des nœuds

6.5 Synthèse du chapitre

L'architecture matérielle du système repose sur des nœuds ESP32 autonomes, chacun équipé d'un bouton Andon et de deux LED indicatrices. Le réseau Wi-Fi de l'atelier assure la transmission des données vers un serveur backend. Le coût total du matériel est très compétitif (≈ 96EUR pour 8 nœuds), démontrant la pertinence de cette approche par rapport aux solutions commerciales.

Architecture logicielle

Ce chapitre présente l'architecture logicielle du système Andon, en décrivant l'organisation du code source, les modules du serveur backend et de l'application frontend, ainsi que les mécanismes d'intégration.

7.1 Organisation globale du code

Le code source du système est organisé en deux parties principales :

- **Backend (serveur Node.js)** : répertoire `server/` contenant le serveur Express, le client MQTT et les routes API.
- **Frontend (Angular)** : répertoire `client/` contenant l'application Angular.

Organisation du code source du système

(Figure à insérer)

Figure 7.1. Organisation du code source du système

7.2 Le serveur backend

7.2.1 Point d'entrée : server.js

Le fichier `server.js` constitue le point d'entrée principal du serveur. Il initialise les composants suivants :

1. Le serveur Express avec ses middleware (JSON, CORS, authentification).
2. Le serveur Socket.IO pour les communications temps réel.
3. La connexion au broker MQTT (Mosquitto).
4. La connexion à la base de données PostgreSQL.
5. Le démarrage du serveur HTTP sur le port 3000.

7.2.2 Structure modulaire

Le serveur suit une architecture modulaire :

- `routes/` : définition des endpoints API (REST).
- `middleware/` : middleware d'authentification JWT, gestion des erreurs.
- `models/` : modèles de données et requêtes SQL.
- `services/` : logique métier (traitement MQTT, génération de rapports).
- `config/` : configuration du serveur (port, bases de données, MQTT).

7.2.3 Connexion MQTT

Le serveur se connecte au broker MQTT au démarrage et s'abonne aux topics pertinents :

```
1  const mqtt = require('mqtt');
2  const client = mqtt.connect('mqtt://localhost:1883', {
3    clientId: 'sews-server',
4    clean: true,
5    reconnectPeriod: 5000
6  });
7
8  client.on('connect', () => {
9    client.subscribe('sews/andon/+/alert');
10   client.subscribe('sews/capteurs/+/status');
11 });
```

Listing 7.1. Connexion et abonnement MQTT

7.2.4 Diffusion Socket.IO

Le serveur utilise Socket.IO pour diffuser les événements en temps réel :

```

1  const io = require('socket.io')(server);
2
3  io.on('connection', (socket) => {
4    socket.on('subscribe', (room) => {
5      socket.join(room);
6    });
7  });
8
9  function broadcastAndonEvent(event) {
10   io.to('dashboard').emit('andon-event', event);
11   io.to('ligne-' + event.ligneId)
12     .emit('ligne-update', event);
13 }
```

Listing 7.2. Diffusion des événements via Socket.IO

7.2.5 API REST

L'API REST expose les endpoints suivants :

Méthode	Endpoint	Description
GET	/api/lignes	Liste des lignes
GET	/api/lignes/ :id	Détail d'une ligne
GET	/api/evenements	Liste des événements
POST	/api/evenements	Créer un événement
PUT	/api/evenements/ :id	Modifier un événement
GET	/api/rapports	Statistiques et rapports
POST	/api/auth/login	Authentification
GET	/api/users	Liste des utilisateurs

Table 7.1. Endpoints de l'API REST

7.3 L'application frontend Angular

7.3.1 Modules fonctionnels

L'application Angular est structurée en modules fonctionnels :

- **AppModule** : module racine avec le routeur et la configuration générale.
- **DashboardModule** : tableau de bord principal avec vue d'ensemble des lignes.
- **LignesModule** : liste et détail des lignes de production.
- **EvenementsModule** : historique et suivi des pannes.
- **RapportsModule** : génération de rapports et statistiques.
- **AdminModule** : administration des utilisateurs.
- **AuthModule** : authentification et gestion des sessions.

7.3.2 Services Angular

Les services assurent la communication avec le backend :

```
1 @Injectable({ providedIn: 'root' })
2 export class SocketService {
3   private socket: any;
4
5   connect() {
6     this.socket = io(environment.apiUrl);
7   }
8
9   subscribeToEvent(event: string, callback: Function) {
10    this.socket.on(event, callback);
11  }
12
13  joinRoom(room: string) {
14    this.socket.emit('subscribe', room);
15  }
16 }
```

Listing 7.3. Service de communication temps réel

7.3.3 Routing

Le routing de l'application définit les paths suivants :

7.4 Inté-gration des composants

7.4.1 Flux de communication

La communication entre les composants suit le flux suivant :

Path	Composant
/	DashboardComponent
/lignes	LignesComponent
/lignes/ :id	LigneDetailComponent
/evenements	EvenementsComponent
/rapports	RapportsComponent
/admin	AdminComponent
/login	LoginComponent

Table 7.2. Routes de l'application Angular

1. L'ESP32 publie un message MQTT.
2. Le broker MQTT redistribue aux abonnés.
3. Le serveur Node.js reçoit le message via le client MQTT.
4. Le serveur traite et enregistre l'événement.
5. Le serveur diffuse via Socket.IO.
6. Le frontend Angular reçoit la notification.
7. Le composant Angular met à jour l'affichage.

7.4.2 Sécurité

La sécurité est assurée par :

- **Authentification JWT** : tokens signés avec expiration.
- **Mots de passe hashés** : algorithme bcrypt.
- **HTTPS** : chiffrement des communications.
- **Validation des entrées** : vérification côté serveur.
- **Journalisation** : logs de toutes les actions critiques.

7.5 Synthèse du chapitre

L'architecture logicielle du système repose sur un serveur Node.js (Express, MQTT, Socket.IO), une base de données PostgreSQL et une application Angular. L'intégration est assurée par le protocole MQTT pour les capteurs et Socket.IO pour le temps réel. La structure modulaire du code facilite la maintenance et l'évolution du système.

Conception de la base de données

Ce chapitre présente la conception du modèle de données du système Andon, depuis l'analyse des besoins en données jusqu'à l'implémentation SQL dans PostgreSQL.

8.1 Méthodologie de conception

8.1.1 Approche adoptée

La conception de la base de données suit une approche descendante :

1. Identification des entités et de leurs attributs.
2. Définition des relations entre les entités.
3. Normalisation du modèle (3FN minimum).
4. Implémentation du schéma SQL.
5. Ajout des index et optimisations.

8.1.2 Entités principales

L'analyse des besoins a permis d'identifier les entités suivantes :

- **Ligne** : représente une ligne de production de l'atelier.
- **Evenement** : représente un événement Andon (panne, anomalie).
- **Utilisateur** : représente un utilisateur du système.
- **Notification** : représente une notification envoyée.
- **Capteur** : représente un nœud ESP32 connecté.

8.2 Modèle Entité-Association

8.2.1 Diagramme EA

Le modèle Entité-Association décrit les entités, leurs attributs et les relations entre elles :

- **Ligne – Evenement** : une ligne peut générer plusieurs événements (relation 1 :N).
- **Utilisateur – Evenement** : un utilisateur peut créer ou gérer plusieurs événements (relation 1 :N).
- **Ligne – Capteur** : une ligne est associée à un capteur (relation 1 :1).
- **Evenement – Notification** : un événement peut déclencher plusieurs notifications (relation 1 :N).

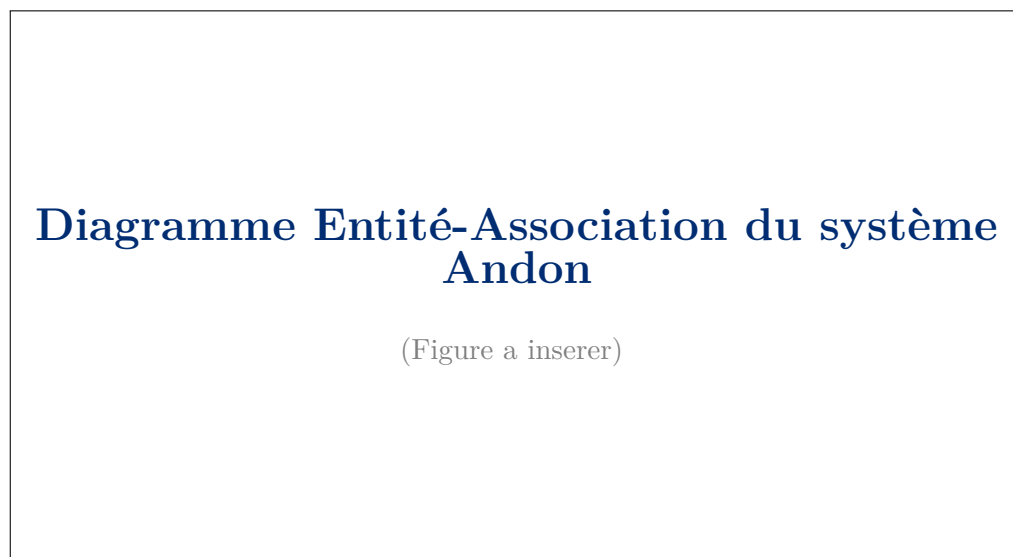


Figure 8.1. Diagramme Entité-Association du système Andon

8.2.2 Définition des attributs

Entité Ligne

Entité Evenement

8.3 Modèle relationnel

8.3.1 Schéma relationnel

Le passage du modèle EA au modèle relationnel produit les tables suivantes :

```
1 Ligne (id, nom, numero, statut, date_creation)
2 Evenement (id, ligne_id, type, statut,
```

Attribut	Type	Description
id	SERIAL (PK)	Identifiant unique
nom	VARCHAR(100)	Nom de la ligne
numero	INTEGER	Numéro de la ligne
statut	VARCHAR(20)	Statut actuel
date_creation	TIMESTAMP	Date de création

Table 8.1. Attributs de l'entité Ligne

Attribut	Type	Description
id	SERIAL (PK)	Identifiant unique
ligne_id	INTEGER (FK)	Référence à la ligne
type	VARCHAR(50)	Type d'événement
statut	VARCHAR(20)	En cours / Ré-solu
description	TEXT	Description détaillée
date_debut	TIMESTAMP	Heure de détection
date_fin	TIMESTAMP	Heure de ré-solution
technicien_id	INTEGER (FK)	Technicien assigné
compte_rendu	TEXT	Compte-rendu d'intervention

Table 8.2. Attributs de l'entité Evenement


```
3         description, date_debut, date_fin,  
4         technicien_id, compte_rendu)  
5 Utilisateur (id, nom, prenom, email, role,  
6             mot_de_passe, date_creation)  
7 Notification (id, evenement_id, destinataire_id,  
8             type, contenu, date_envoi, statut)  
9 Capteur (id, ligne_id, mac_address,  
10         firmware_version, date_installation)
```

Listing 8.1. Schéma relationnel

8.3.2 Règles d'intégrité

Les contraintes d'intégrité suivantes sont appliquées :

- **Clés primaires** : chaque table possède un identifiant unique auto-incrémenté (SERIAL).
- **Clés étrangères** : les références entre tables sont contrôlées par des contraintes FOREIGN KEY.
- **NOT NULL** : les attributs critiques ne peuvent être nuls.
- **UNIQUE** : l'adresse e-mail des utilisateurs doit être unique.
- **CHECK** : les statuts doivent respecter les valeurs définies.

8.4 Implémentation SQL

8.4.1 Création des tables

```
1 CREATE TABLE lignes (  
2     id SERIAL PRIMARY KEY,  
3     nom VARCHAR(100) NOT NULL,  
4     numero INTEGER UNIQUE NOT NULL,  
5     statut VARCHAR(20) DEFAULT 'active'  
6     CHECK (statut IN (  
7         'active', 'inactive', 'en_panne'  
8     )),  
9     date_creation TIMESTAMP  
10    DEFAULT CURRENT_TIMESTAMP  
11 );  
12  
13 CREATE TABLE utilisateurs (  
14     id SERIAL PRIMARY KEY,
```

```
15     nom VARCHAR(100) NOT NULL ,
16     prenom VARCHAR(100) NOT NULL ,
17     email VARCHAR(255) UNIQUE NOT NULL ,
18     role VARCHAR(50) NOT NULL
19         CHECK (role IN (
20             'admin', 'technicien', 'chef', 'opérateur'
21         )),
22     mot_de_passe VARCHAR(255) NOT NULL ,
23     date_creation TIMESTAMP
24         DEFAULT CURRENT_TIMESTAMP
25 );
26
27 CREATE TABLE evenements (
28     id SERIAL PRIMARY KEY ,
29     ligne_id INTEGER NOT NULL
30         REFERENCES lignes(id),
31     type VARCHAR(50) NOT NULL ,
32     statut VARCHAR(20) DEFAULT 'en_cours'
33         CHECK (statut IN ('en_cours', 'resolu')),
34     description TEXT ,
35     date_debut TIMESTAMP
36         DEFAULT CURRENT_TIMESTAMP ,
37     date_fin TIMESTAMP ,
38     technicien_id INTEGER
39         REFERENCES utilisateurs(id),
40     compte_rendu TEXT
41 );
42
43 CREATE TABLE notifications (
44     id SERIAL PRIMARY KEY ,
45     evenement_id INTEGER
46         REFERENCES evenements(id),
47     destinataire_id INTEGER
48         REFERENCES utilisateurs(id),
49     type VARCHAR(50) NOT NULL ,
50     contenu TEXT ,
51     date_envoi TIMESTAMP
52         DEFAULT CURRENT_TIMESTAMP ,
53     statut VARCHAR(20) DEFAULT 'envoyee'
54 );
```

Listing 8.2. Création des tables principales

8.4.2 Index de performance

Pour garantir des performances optimales, les index suivants sont créés :

```
1 CREATE INDEX idx_evenements_ligne
2   ON evenements(ligne_id);
3 CREATE INDEX idx_evenements_date
4   ON evenements(date_debut);
5 CREATE INDEX idx_evenements_statut
6   ON evenements(statut);
7 CREATE INDEX idx_evenements_date_ligne
8   ON evenements(date_debut, ligne_id);
9 CREATE INDEX idx_notifications_dest
10  ON notifications(destinataire_id);
```

Listing 8.3. Index de performance

8.4.3 Données de test

Des données de test ont été insérées pour valider le système :

```
1 INSERT INTO lignes (nom, numero, statut) VALUES
2 ('Ligne T1', 1, 'active'),
3 ('Ligne T2', 2, 'active'),
4 ('Ligne T3', 3, 'active'),
5 ('Ligne T4', 4, 'active'),
6 ('Ligne T5', 5, 'active'),
7 ('Ligne T6', 6, 'active'),
8 ('Ligne T7', 7, 'active'),
9 ('Ligne T8', 8, 'active');
```

Listing 8.4. Insertion des lignes de test

8.5 Gestion des volumes de données

8.5.1 Stratégie de rétention

Étant donné le volume croissant d'événements générés, une stratégie de rétention est mise en place :

- Les événements détaillés sont conservés pendant 12 mois.
- Les données agrégées sont conservées indéfiniment.

- Un job de nettoyage périodique supprime les données anciennes.

8.5.2 Partitionnement temporel

Pour les volumes importants, le partitionnement temporel est envisagé :

```
1 CREATE TABLE evenements (  
2     ...  
3 ) PARTITION BY RANGE (date_debut);  
4  
5 CREATE TABLE evenements_2026_q1  
6     PARTITION OF evenements  
7     FOR VALUES FROM ('2026-01-01')  
8     TO ('2026-04-01');
```

Listing 8.5. Partitionnement temporel

8.6 Synthèse du chapitre

La conception de la base de données suit une approche rigoureuse allant du modèle EA au schéma SQL. Le modèle comprend cinq entités principales : Ligne, Evenement, Utilisateur, Notification et Capteur. L'implémentation dans PostgreSQL intègre les contraintes d'intégrité, les index de performance et une stratégie de gestion des volumes de données adaptée aux besoins du système Andon.

Architecture de communication

Ce chapitre analyse les protocoles et les flux de données du système Andon, en justifiant les choix de communication et en décrivant les mécanismes d'interconnexion entre les différents composants.

9.1 Protocoles de communication utilisés

9.1.1 MQTT : protocole principal

MQTT (Message Queuing Telemetry Transport) est le protocole principal du système, assurant la communication entre les capteurs ESP32 et le serveur backend [4].

Justification du choix

Le choix de MQTT est justifié par :

- Légèreté des messages : en-têtes minimaux (2 octets), adapté aux microcontrôleurs.
- Le modèle publish/subscribe : dé-couplage entre producteurs et consommateurs de données.
- Les niveaux de QoS : garantie de livraison configurable selon la criticité.
- La reconnexion automatique : le client se reconnecte après une déconnexion.
- La large adoption : supporté par toutes les plateformes IoT.

Configuration du broker

Le broker MQTT Mosquitto est configuré pour le système Andon :

```
1 listener 1883 localhost
2 listener 8883
3 certfile /etc/mosquitto/certs/server.crt
4 keyfile /etc/mosquitto/certs/server.key
5 allow_anonymous false
6 password_file /etc/mosquitto/passwd
```

```
7 max_connections -1
8 max_inflight_messages 20
9 max_queued_messages 1000
```

Listing 9.1. Configuration du broker Mosquitto

9.1.2 HTTP/HTTPS : API REST

Le protocole HTTP est utilisé pour les communications entre le frontend et le backend via l'API REST :

- Les requêtes GET pour la lecture de données.
- Les requêtes POST pour la création de ressources.
- Les requêtes PUT pour la mise à jour de ressources.
- Les requêtes DELETE pour la suppression de ressources.

En environnement de production, le HTTPS est obligatoire pour chiffrer les communications.

9.1.3 WebSocket (Socket.IO)

Socket.IO est utilisé pour les communications temps réel bidirectionnelles entre le serveur et le frontend :

- Diffusion instantanée des événements Andon.
- Mise à jour de l'état des lignes en direct.
- Notifications visuelles en temps réel.
- Fallback automatique sur HTTP long-polling si WebSocket n'est pas disponible.

9.2 Structure des topics MQTT

9.2.1 Hierarchie des topics

La hiérarchie des topics MQTT reflète l'organisation physique de l'atelier :

```
1 # Alertes Andon (du capteur au serveur)
2 sews/andon/1/alert
3 sews/andon/2/alert
4 ...
5 sews/andon/8/alert
6
```

```
7 # Status des capteurs
8 sews/capteurs/1/status
9 sews/capteurs/2/status
10 ...
11 sews/capteurs/8/status
12
13 # Commandes retour (du serveur au capteur)
14 sews/command/1/led
15 sews/command/2/led
16 ...
17 sews/command/8/led
```

Listing 9.2. Hiérarchie des topics MQTT

9.2.2 Format des messages

Les messages MQTT utilisent le format JSON pour faciliter le traitement :

```
1 {
2   "ligneId": 1,
3   "timestamp": "2026-04-15T10:30:00.000Z",
4   "type": "panne_electrique",
5   "urgence": "haute",
6   "opérateur": "op_t1_01",
7   "description": "Court-circuit detecte
8     sur testeur 3"
9 }
```

Listing 9.3. Format du message d'alerte Andon

9.3 Flux de données détaillé

9.3.1 Flux d'un événement Andon

Le parcours complet d'un événement Andon suit les étapes suivantes :

1. **Détection** : l'opérateur appuie sur le bouton physique.
2. **Traitement local** : l'ESP32 détecte l'appui, applique un débounce de 200 ms et génère un message JSON.
3. **Publication MQTT** : le message est publié sur le topic `sews/andon/{ligne_id}/alert` avec QoS 1.
4. **Routing** : le broker MQTT redistribue le message aux abonnés (serveur ba-

ckend).

5. **Traitement serveur** : le serveur Node.js reçoit le message, valide les données et crée un enregistrement dans PostgreSQL.
6. **Diffusion temps réel** : le serveur diffuse l'événement via Socket.IO aux clients connectés.
7. **Affichage** : le frontend Angular reçoit la notification et met à jour le tableau de bord.
8. **Notification** : un e-mail est envoyé au technicien de garde.
9. **Indicateur visuel** : le serveur publie une commande sur `sews/command/{ligne_id}/led` pour allumer la LED rouge.

9.3.2 Flux de résolution

Lorsqu'un technicien réesout une panne :

1. Le technicien accède à l'événement dans le frontend.
2. Il saisit le compte-rendu d'intervention et valide la ré-solution.
3. Le frontend envoie une requête PUT à l'API REST (`PUT /api/evenements/:id`).
4. Le serveur met à jour l'enregistrement dans PostgreSQL (`statut = resolu, date_fin = now`).
5. Le serveur publie la commande de ré-solution sur MQTT.
6. L'ESP32 reçoit la commande et éteint la LED rouge, allume la LED verte.
7. Le frontend affiche la ligne comme ré-solue.
8. Une notification de ré-solution est envoyée.

9.4 Gestion de la sécurité des communications

9.4.1 Sécurité MQTT

La sécurité de la communication MQTT est assurée par :

- **Authentication** : chaque client MQTT doit fournir un identifiant et un mot de passe.
- **TLS** : chiffrement des communications sur le port 8883.
- **ACL** : contrôle d'accès par topic (chaque nœud ne publie que sur son topic).
- **Persistence** : les messages QoS 1 sont conservés jusqu'à réception de l'accusé.

9.4.2 Sécurité HTTP

La sécurité de l'API REST est assurée par :

- **HTTPS** : chiffrement TLS des communications.
- **JWT** : tokens d'authentification signés avec expiration.
- **Validation** : vérification des données d'entrée.
- **Rate limiting** : limitation du nombre de requêtes.

9.5 Gestion des erreurs et tolérance aux pannes

9.5.1 Stratégie de reconnexion

Le client MQTT intègre un mécanisme de reconnexion automatique :

- Tentative de reconnexion toutes les 5 secondes en cas de déconnexion.
- Backoff exponentiel avec un délai maximum de 60 secondes.
- Conservation des messages en attente pendant la déconnexion.
- Re-souscription aux topics lors de la reconnexion.

9.5.2 Persistance des données

En cas de défaut du serveur :

- Les messages MQTT non traités sont conservés dans le broker (QoS 1).
- PostgreSQL assure la durabilité des données enregistrées (WAL).
- Le frontend affiche les données en cache en attendant la reconnexion.

9.6 Synthèse du chapitre

L'architecture de communication du système repose sur trois protocoles complémentaires : MQTT pour la couche capteur, HTTP/REST pour l'API, et Socket.IO pour le temps réel. La hiérarchie des topics MQTT, les flux de données détaillés et les mécanismes de sécurité forment un système cohérent et fiable, capable de gérer les contraintes de l'environnement industriel.

Implémentation du système

Ce chapitre présente l'implémentation concrète du système Andon, en décrivant les étapes de développement, l'environnement de travail et la mise en oeuvre de chaque composant.

10.1 Environnement de développement

10.1.1 Outils utilisés

Catégorie	Outil
Système d'exploitation	Windows 10/11
Editeur de code	Visual Studio Code
Serveur backend	Node.js v18 LTS + npm
Base de données	PostgreSQL 15
Broker MQTT	Mosquitto 2.0
Framework frontend	Angular 16
Microcontrôleur	ESP32 (Arduino IDE)
Gestion de version	Git + GitHub

Table 10.1. Environnement de développement

10.1.2 Installation des dépendances

L'installation du projet se fait en quelques commandes :

```
1 # Cloner le depot
2 git clone https://github.com/sews/andon-iiot.git
3
4 # Installer les d\'ependances du serveur
5 cd server
6 npm install
```

```
7
8 # Installer les d\'ependances du client
9 cd ../client
10 npm install
```

Listing 10.1. Installation du backend

10.2 Mise en place de l'infrastructure

10.2.1 Installation de PostgreSQL

La base de données est créée en exécutant le script SQL d'initialisation :

```
1 # Creation de la base
2 createdb sews_andon
3
4 # Execution du script de schema
5 psql -d sews_andon -f schema.sql
6
7 # Insertion des donnees de test
8 psql -d sews_andon -f seed.sql
```

Listing 10.2. Initialisation de la base de données

10.2.2 Configuration de Mosquitto

Le broker MQTT est installé et configuré selon les besoins du système :

```
1 # Installation (Ubuntu)
2 sudo apt install mosquitto
3
4 # Demarrage du service
5 sudo systemctl start mosquitto
6 sudo systemctl enable mosquitto
7
8 # Test de connexion
9 mosquitto_pub -t "test/topic" -m "hello"
```

Listing 10.3. Installation de Mosquitto

10.3 Développement du firmware ESP32

10.3.1 Programme principal

Le firmware de l'ESP32 est développé en Arduino (C/C++) et suit la logique suivante :

```
1 void setup() {
2     Serial.begin(115200);
3     pinMode(BOUTON_PIN, INPUT);
4     pinMode(LED_VERTE, OUTPUT);
5     pinMode(LED_ROUGE, OUTPUT);
6
7     connectWiFi();
8     connectMQTT();
9
10    digitalWrite(LED_VERTE, HIGH);
11 }
12
13 void loop() {
14     if (!mqttClient.connected()) {
15         reconnectMQTT();
16     }
17     mqttClient.loop();
18
19     if (digitalRead(BOUTON_PIN) == HIGH) {
20         delay(200); // Debounce
21         envoyerAlerte();
22         digitalWrite(LED_VERTE, LOW);
23         digitalWrite(LED_ROUGE, HIGH);
24     }
25 }
```

Listing 10.4. Boucle principale du firmware ESP32

10.3.2 Fonction d'envoi d'alerte

```
1 void envoyerAlerte() {
2     StaticJsonDocument<200> doc;
3     doc["ligneId"] = LIGNE_ID;
4     doc["timestamp"] = getTimestampTimestamp();
5     doc["type"] = "panne";
6     doc["urgence"] = "haute";
7
8     char buffer[200];
9     serializeJson(doc, buffer);
10 }
```

```
11 String topic = "sews/andon/"
12     + String(LIGNE_ID) + "/alert";
13 mqttClient.publish(
14     topic.c_str(), buffer, true
15 );
16 }
```

Listing 10.5. Envoi d'une alerte Andon via MQTT

10.4 Développement du serveur backend

10.4.1 Structure du projet

Le serveur backend suit la structure suivante :

```
1 server/
2   |-- server.js           Point d'entree
3   |-- package.json       Dependances
4   |-- config/
5   |   |-- db.js          Configuration PostgreSQL
6   |   |-- mqtt.js        Configuration MQTT
7   |-- routes/
8   |   |-- evenements.js  Routes API evenements
9   |   |-- lignes.js      Routes API lignes
10  |   |-- users.js        Routes API utilisateurs
11  |-- middleware/
12  |   |-- auth.js         Authentification JWT
13  |-- models/
14  |   |-- Evenement.js    Modele evenement
15  |   |-- Ligne.js        Modele ligne
```

Listing 10.6. Arborescence du serveur backend

10.4.2 Démarrage du serveur

```
1 const express = require('express');
2 const http = require('http');
3 const { Server } = require('socket.io');
4 const mqtt = require('mqtt');
5 const { Pool } = require('pg');
6
7 const app = express();
```

```
8  const server = http.createServer(app);
9  const io = new Server(server);
10
11  // Connexion PostgreSQL
12  const pool = new Pool({
13    host: 'localhost',
14    port: 5432,
15    database: 'sews_andon',
16    user: 'sews_admin',
17    password: process.env.DB_PASS
18  });
19
20  // Connexion MQTT
21  const mqttClient = mqtt.connect(
22    'mqtt://localhost:1883'
23  );
24
25  server.listen(3000, () => {
26    console.log('Serveur démarre sur :3000');
27  });
```

Listing 10.7. Démarrage du serveur backend

10.5 Développement du frontend Angular

10.5.1 Structure du projet

```
1  client/
2    |-- src/
3    |   |-- app/
4    |   |   |-- components/
5    |   |   |   |-- dashboard/
6    |   |   |   |-- lignes/
7    |   |   |   |-- evenements/
8    |   |   |-- services/
9    |   |   |   |-- api.service.ts
10   |   |   |   |-- socket.service.ts
11   |   |   |-- models/
12   |   |   |-- app-routing.module.ts
13   |   |-- assets/
14   |   |-- environments/
15   |-- angular.json
16   |-- package.json
```

Listing 10.8. Arborescence du frontend Angular

10.6 Tests et validation

10.6.1 Tests unitaires

Des tests unitaires ont été réalisés pour chaque composant :

- Tests de l'API REST (CRUD opérations).
- Tests de la connexion MQTT.
- Tests de la base de données (requêtes SQL).
- Tests des composants Angular (navigation, affichage).

10.6.2 Tests d'intégration

Les tests d'intégration vérifient le bon fonctionnement de l'ensemble :

1. Envoi d'un message MQTT de test.
2. Vérification de la création de l'événement dans PostgreSQL.
3. Vérification de la diffusion via Socket.IO.
4. Vérification de l'affichage dans le frontend.

10.7 Synthèse du chapitre

L'implémentation du système a mobilisé des compétences dans différents domaines : programmation embarquée (Arduino/C++), développement backend (Node.js/Express), développement frontend (Angular/TypeScript) et administration de base de données (PostgreSQL). Chaque composant a été développé, testé et intégré de manière cohérente.

Chapitre 11

Implémentation du backend

Ce chapitre développe en détail l'implémentation du serveur backend du système Andon, en présentant la structure du code, les middleware, les routes API et les services métier.

11.1 Point d'entrée : server.js

11.1.1 Initialisation du serveur

Le fichier `server.js` constitue le point d'entrée principal. Il configure et initialise l'ensemble des composants :

```
1  const express = require('express');
2  const cors = require('cors');
3  const http = require('http');
4  const { Server } = require('socket.io');
5  const mqtt = require('mqtt');
6  const { Pool } = require('pg');
7
8  const app = express();
9  const server = http.createServer(app);
10
11  // Middleware
12  app.use(cors());
13  app.use(express.json());
14
15  // Socket.IO
16  const io = new Server(server, {
17    cors: { origin: '*' }
18  });
19
20  // PostgreSQL
21  const pool = new Pool({
22    host: process.env.DB_HOST,
```



```
23   port: process.env.DB_PORT,
24   database: process.env.DB_NAME,
25   user: process.env.DB_USER,
26   password: process.env.DB_PASS,
27   max: 20
28 });
```

Listing 11.1. Initialisation du serveur Express

11.1.2 Connexion MQTT et gestion des messages

```
1  const mqttClient = mqtt.connect(
2    process.env.MQTT_URL, {
3      clientId: 'sews-server',
4      clean: true,
5      reconnectPeriod: 5000
6    }
7  );
8
9  mqttClient.on('connect', () => {
10    console.log('MQTT connecte');
11    mqttClient.subscribe('sews/andon/+/alert');
12    mqttClient.subscribe('sews/capteurs/+/status');
13  });
14
15  mqttClient.on('message', async (topic, msg) => {
16    const parts = topic.split('/');
17    const type = parts[1];
18    const ligneId = parts[2];
19    const data = JSON.parse(msg.toString());
20
21    if (type === 'andon') {
22      await handleAndonAlert(ligneId, data);
23    } else if (type === 'capteurs') {
24      await handleCapteurStatus(ligneId, data);
25    }
26  });
```

Listing 11.2. Gestion des messages MQTT

11.1.3 Traitement des alertes Andon

```
1  async function handleAndonAlert(ligneId, data) {
```

```
2  const query = `
3      INSERT INTO evenements
4          (ligne_id, type, description, date_debut)
5      VALUES ($1, $2, $3, NOW())
6      RETURNING *
7  `;
8  const values = [
9      ligneId,
10     data.type || 'panne',
11     data.description || ''
12 ];
13
14 const result = await pool.query(query, values);
15 const evenement = result.rows[0];
16
17 // Diffusion Socket.IO
18 io.to('dashboard').emit('andon-event', {
19     evenement,
20     ligneId: parseInt(ligneId)
21 });
22
23 // Commande LED rouge
24 const cmdTopic = `sews/command/${ligneId}/led`;
25 mqttClient.publish(cmdTopic,
26     JSON.stringify({ rouge: true, verte: false })
27 );
28 }
```

Listing 11.3. Traitement d'une alerte Andon

11.2 Middleware d'authentification

11.2.1 Génération et validation des tokens JWT

```
1  const jwt = require('jsonwebtoken');
2  const SECRET = process.env.JWT_SECRET;
3
4  function genererToken(user) {
5      return jwt.sign(
6          { id: user.id, role: user.role },
7          SECRET,
8          { expiresIn: '24h' }
9      );
10 }
```

```
10 }
11
12 function authMiddleware(req, res, next) {
13   const token = req.headers.authorization
14     ?.split(' ')[1];
15
16   if (!token) {
17     return res.status(401).json({
18       error: 'Token requis'
19     });
20   }
21
22   try {
23     const decoded = jwt.verify(token, SECRET);
24     req.user = decoded;
25     next();
26   } catch (err) {
27     return res.status(401).json({
28       error: 'Token invalide'
29     });
30   }
31 }
```

Listing 11.4. Middleware JWT

11.3 Routes API

11.3.1 Route d'authentification

```
1  const bcrypt = require('bcrypt');
2
3  app.post('/api/auth/login', async (req, res) => {
4    const { email, password } = req.body;
5
6    const result = await pool.query(
7      'SELECT * FROM utilisateurs WHERE email=$1',
8      [email]
9    );
10
11    if (result.rows.length === 0) {
12      return res.status(401).json({
13        error: 'Identifiants incorrects'
14      });
15    }
16  });
```

```
15   }
16
17   const user = result.rows[0];
18   const valid = await bcrypt.compare(
19     password, user.mot_de_passe
20   );
21
22   if (!valid) {
23     return res.status(401).json({
24       error: 'Identifiants incorrects'
25     });
26   }
27
28   const token = genererToken(user);
29   res.json({ token, user: {
30     id: user.id,
31     nom: user.nom,
32     prenom: user.prenom,
33     role: user.role
34   }});
35 });
```

Listing 11.5. Route de connexion

11.3.2 Routes CRUD des événements

```
1 // GET /api/evenements
2 app.get('/api/evenements',
3   authMiddleware, async (req, res) => {
4     const { page = 1, limite = 20 } = req.query;
5     const offset = (page - 1) * limite;
6
7     const result = await pool.query(`
8       SELECT e.*, l.nom as ligne_nom
9       FROM evenements e
10      JOIN lignes l ON e.ligne_id = l.id
11      ORDER BY e.date_debut DESC
12      LIMIT $1 OFFSET $2
13    `, [limite, offset]);
14
15    res.json(result.rows);
16  });
17
18 // PUT /api/evenements/:id
```

```

19 app.put('/api/evenements/:id',
20   authMiddleware, async (req, res) => {
21     const { statut, compte_rendu } = req.body;
22
23     const result = await pool.query('
24       UPDATE evenements
25       SET statut = $1,
26         compte_rendu = $2,
27         date_fin = CASE WHEN $1='resolu'
28           THEN NOW() ELSE date_fin END
29       WHERE id = $3
30       RETURNING *
31     ', [statut, compte_rendu, req.params.id]);
32
33     // Diffusion Socket.IO
34     io.to('dashboard').emit('resolution', {
35       evenement: result.rows[0]
36     });
37
38     // Commande LED verte
39     const ligneId = result.rows[0].ligne_id;
40     mqttClient.publish(
41       'sews/command/${ligneId}/led',
42       JSON.stringify({ rouge: false, verte: true })
43     );
44
45     res.json(result.rows[0]);
46   });

```

Listing 11.6. Routes des événements

11.3.3 Route de rapports

```

1 app.get('/api/rapports',
2   authMiddleware, async (req, res) => {
3     const { dateDebut, dateFin } = req.query;
4
5     const stats = await pool.query('
6       SELECT
7         l.nom as ligne,
8         COUNT(*) as nb_evenements,
9         AVG(EXTRACT(EPOCH FROM
10           (COALESCE(date_fin, NOW())
11             - date_debut))) as duree_moyenne,

```

```

12     COUNT(CASE WHEN statut='resolu'
13              THEN 1 END) as nb_resolus
14   FROM evenements e
15   JOIN lignes l ON e.ligne_id = l.id
16   WHERE e.date_debut BETWEEN $1 AND $2
17   GROUP BY l.nom
18   ORDER BY nb_evenements DESC
19   ', [dateDebut, dateFin]);
20
21   res.json(stats.rows);
22 });

```

Listing 11.7. Route de rapports

11.4 Gestion des notifications

11.4.1 Notifications par e-mail

Le serveur intègre un module d'envoi d'e-mails via Nodemailer :

```

1  const nodemailer = require('nodemailer');
2
3  const transporter = nodemailer.createTransport({
4    host: 'smtp.sews.ma',
5    port: 587,
6    secure: false,
7    auth: {
8      user: process.env.SMTP_USER,
9      pass: process.env.SMTP_PASS
10   }
11 });
12
13 async function envoyerNotification(event, user) {
14   await transporter.sendMail({
15     from: '"Andon SEWS" <andon@sews.ma>',
16     to: user.email,
17     subject: 'Alerte panne - Ligne ${event.ligne_id}',
18     html: '
19       <h2>Alerte Andon</h2>
20       <p>Ligne: ${event.ligne_id}</p>
21       <p>Type: ${event.type}</p>
22       <p>Heure: ${event.date_debut}</p>
23     '

```

```
24     });  
25 }
```

Listing 11.8. Envoi de notification e-mail

11.5 Synthèse du chapitre

Le serveur backend du système Andon est implémenté en Node.js avec Express. Il intègre la connexion MQTT, la persistance PostgreSQL, l'authentification JWT, les routes API REST et la diffusion temps réel via Socket.IO. Le code est organisé de manière modulaire pour faciliter la maintenance et les évolutions futures.

Implémentation du frontend

Ce chapitre présente l'implémentation de l'application frontend Angular du système Andon, en décrivant la structure du projet, les composants principaux, les services et les mécanismes de communication avec le backend.

12.1 Initialisation du projet Angular

12.1.1 Création du projet

Le projet Angular a été initialisé avec Angular CLI :

```
1 ng new client --routing --style=scss
2 cd client
3 ng add @angular/material
4 npm install socket.io-client
5 npm install chart.js ng2-charts
```

Listing 12.1. Création du projet Angular

12.2 Composants principaux

12.2.1 Le tableau de bord (Dashboard)

Le DashboardComponent est le composant central de l'application. Il affiche la vue d'ensemble des 8 lignes de production :

```
1 @Component({
2   selector: 'app-dashboard',
3   templateUrl: './dashboard.component.html'
4 })
5 export class DashboardComponent
6   implements OnInit, OnDestroy {
7
```



```
8   lignes: Ligne[] = [];  
9   evenementsRecents: Evenement[] = [];  
10  private subscription: any;  
11  
12  constructor(  
13    private apiService: ApiService,  
14    private socketService: SocketService  
15  ) {}  
16  
17  ngOnInit() {  
18    this.chargerLignes();  
19    this.ecouterEvenements();  
20  }  
21  
22  chargerLignes() {  
23    this.apiService.getLignes()  
24      .subscribe(data => {  
25        this.lignes = data;  
26      });  
27  }  
28  
29  écouterEvenements() {  
30    this.subscription =  
31      this.socketService  
32        .on('andon-event')  
33        .subscribe(event => {  
34          this.mettreAJourLigne(event);  
35          this.ajouterEvenement(event);  
36        });  
37  }  
38  
39  ngOnDestroy() {  
40    this.subscription?.unsubscribe();  
41  }  
42 }
```

Listing 12.2. Composant Dashboard

12.2.2 Le template du tableau de bord

```
1 <div class="dashboard">  
2   <h1>Tableau de bord</h1>  
3  
4   <div class="lignes-grid">
```

```

5      <div *ngFor="let ligne of lignes"
6          class="ligne-card"
7          [class.en-panne]="ligne.statut==='en_panne'"
8          [class.active]="ligne.statut==='active'">
9
10         <h3>{{ ligne.nom }}</h3>
11         <span class="statut-badge">
12             {{ ligne.statut }}
13         </span>
14
15         <div *ngIf="ligne.alerteActive"
16             class="alerte">
17             <span class="pulse"></span>
18             PANNE EN COURS
19             <small>{{ ligne.derniereAlerte }}</small>
20         </div>
21     </div>
22 </div>
23
24 <div class="evenements-recents">
25     <h2>Evenements recents</h2>
26     <div *ngFor="let evt of evenementsRecents"
27         class="evt-item">
28         <strong>Ligne {{ evt.ligneId }}</strong>
29         - {{ evt.type }} - {{ evt.timestamp | date }}
30     </div>
31 </div>
32 </div>

```

Listing 12.3. Template du Dashboard

12.2.3 La page de détail d'une ligne

Le `LigneDetailComponent` affiche les informations détaillées d'une ligne et sa chronologie d'événements :

```

1  @Component({
2      selector: 'app-ligne-detail',
3      templateUrl: './ligne-detail.component.html'
4  })
5  export class LigneDetailComponent
6      implements OnInit {
7
8      ligne: Ligne | null = null;

```

```

 9   evenements: Evenement[] = [];
10   ligneId: number = 0;
11
12   constructor(
13     private route: ActivatedRoute,
14     private apiService: ApiService,
15     private socketService: SocketService
16   ) {}
17
18   ngOnInit() {
19     this.ligneId = +this.route.snapshot
20       .paramMap.get('id')!;
21
22     this.apiService
23       .getLigneById(this.ligneId)
24       .subscribe(data => this.ligne = data);
25
26     this.apiService
27       .getEvenementsByLigne(this.ligneId)
28       .subscribe(data => this.evenements = data);
29   }
30 }

```

Listing 12.4. Composant détail de ligne

12.3 Services de communication

12.3.1 Service API REST

```

 1  @Injectable({ providedIn: 'root' })
 2  export class ApiService {
 3    private apiUrl = environment.apiUrl;
 4
 5    constructor(private http: HttpClient) {}
 6
 7    getLignes(): Observable<Ligne[]> {
 8      return this.http.get<Ligne[]>(
 9        `${this.apiUrl}/api/lignes`
10      );
11    }
12
13    getLigneById(id: number): Observable<Ligne> {
14      return this.http.get<Ligne>(

```

```

15     '${this.apiUrl}/api/lignes/${id}'
16   );
17 }
18
19 getEvenements(params?: any):
20   Observable<Evenement []> {
21   return this.http.get<Evenement []>(
22     '${this.apiUrl}/api/evenements',
23     { params }
24   );
25 }
26
27 resoudreEvenement(
28   id: number, data: any
29 ): Observable<Evenement> {
30   return this.http.put<Evenement>(
31     '${this.apiUrl}/api/evenements/${id}',
32     data
33   );
34 }
35
36 login(email: string, password: string):
37   Observable<any> {
38   return this.http.post(
39     '${this.apiUrl}/api/auth/login',
40     { email, password }
41   );
42 }
43 }

```

Listing 12.5. Service API REST

12.3.2 Service Socket.IO (temps réel)

```

1  @Injectable({ providedIn: 'root' })
2  export class SocketService {
3    private socket: any;
4
5    connect() {
6      this.socket = io(environment.apiUrl);
7
8      this.socket.on('connect', () => {
9        console.log('WebSocket connecte');
10       this.socket.emit('subscribe', 'dashboard');

```

```
11     });
12   }
13
14   on(event: string): Observable<any> {
15     return new Observable(observer => {
16       this.socket.on(event, (data: any) => {
17         observer.next(data);
18       });
19     });
20   }
21
22   disconnect() {
23     this.socket?.disconnect();
24   }
25 }
```

Listing 12.6. Service Socket.IO

12.4 Composants de visualisation

12.4.1 Graphiques de tendances

L'application intègre des graphiques pour visualiser les tendances :

- **Graphique de barres** : nombre d'événements par ligne.
- **Graphique circulaire** : répartition par type de panne.
- **Graphique linéaire** : tendance temporelle des pannes.
- **Indicateurs KPI** : temps moyen de réponse, taux de disponibilité.

12.5 Authentification et gestion des sessions

12.5.1 Routeur et gardes de navigation

```
1  const routes: Routes = [
2    { path: 'login', component: LoginComponent },
3    {
4      path: '',
5      component: DashboardComponent,
6      canActivate: [AuthGuard]
7    },
8    {
```

```
 9     path: 'lignes',
10     component: LignesComponent,
11     canActivate: [AuthGuard]
12 },
13 {
14     path: 'lignes/:id',
15     component: LigneDetailComponent,
16     canActivate: [AuthGuard]
17 },
18 {
19     path: 'evenements',
20     component: EvenementsComponent,
21     canActivate: [AuthGuard]
22 },
23 {
24     path: 'rapports',
25     component: RapportsComponent,
26     canActivate: [AuthGuard]
27 }
28 ];
```

Listing 12.7. Configuration du routeur

12.6 Responsive design

L'application est optimisée pour différents terminaux :

- **Desktop** : vue complète avec tous les graphiques et panneaux.
- **Tablette** : disposition en grille adaptée à la taille d'écran.
- **Smartphone** : navigation simplifiée avec vues empilées.

12.7 Synthèse du chapitre

L'application frontend Angular du système Andon offre une interface moderne, réactive et intuitive. Les composants Dashboard, Lignes, Evenements et Rapports couvrent l'ensemble des besoins fonctionnels. La communication temps réel via Socket.IO et l'authentification JWT assurent une utilisation sécurisée et en direct.

Implémentation ESP32

Ce chapitre développe en détail l'implémentation du firmware de l'ESP32 pour le nœud Andon, en présentant le code source, les bibliothèques utilisées et les mécanismes de communication.

13.1 Environnement de développement

13.1.1 Arduino IDE

Le firmware ESP32 est développé avec Arduino IDE, qui offre :

- Un environnement de développement intuitif.
- La compilation et le téléchargement direct vers l'ESP32.
- Un large écosystème de bibliothèques.
- Le débogage sérial pour le suivi d'exécution.

13.1.2 Bibliothèques utilisées

Bibliothèque	Fonction
WiFi.h	Connexion Wi-Fi
PubSubClient	Client MQTT
ArduinoJson	Génération JSON
WiFiClientSecure	Connexion TLS

Table 13.1. Bibliothèques Arduino utilisées

13.2 Code source du firmware

13.2.1 Configuration initiale

```
1  #include <WiFi.h>
2  #include <PubSubClient.h>
3  #include <ArduinoJson.h>
4
5  // Configuration WiFi
6  const char* SSID = "SEWS_IOT";
7  const char* PASSWORD = "****";
8
9  // Configuration MQTT
10 const char* MQTT_SERVER = "192.168.1.100";
11 const int MQTT_PORT = 1883;
12 const char* MQTT_USER = "esp32_ligne1";
13 const char* MQTT_PASS = "****";
14
15 // Configuration ligne
16 const int LIGNE_ID = 1;
17 const char* TOPIC_ALERT =
18     "sews/andon/1/alert";
19 const char* TOPIC_LED =
20     "sews/command/1/led";
21
22 // Broches GPIO
23 const int BOUTON_PIN = 4;
24 const int LED_VERTE = 16;
25 const int LED_ROUGE = 17;
26
27 WiFiClient wifiClient;
28 PubSubClient mqtt(wifiClient);
29
30 unsigned long dernierAppui = 0;
31 const unsigned long DEBOUNCE = 200;
```

Listing 13.1. Configuration et constantes

13.2.2 Connexion WiFi et MQTT

```
1  void connectWiFi() {
2      Serial.print("Connexion WiFi...");
3      WiFi.begin(SSID, PASSWORD);
4
5      while (WiFi.status() != WL_CONNECTED) {
6          delay(500);
7          Serial.print(".");
```



```
8     }
9
10    Serial.println(" OK");
11    Serial.println(WiFi.localIP());
12 }
13
14 void connectMQTT() {
15     mqtt.setServer(MQTT_SERVER, MQTT_PORT);
16     mqtt.setCallback(callbackMQTT);
17
18     while (!mqtt.connected()) {
19         Serial.print("Connexion MQTT...");
20
21         if (mqtt.connect(
22             "esp32_ligne1", MQTT_USER, MQTT_PASS
23         )) {
24             Serial.println(" OK");
25             mqtt.subscribe(TOPIC_LED);
26         } else {
27             Serial.println(" echec");
28             delay(5000);
29         }
30     }
31 }
```

Listing 13.2. Connexion WiFi et MQTT

13.2.3 Boucle principale

```
1 void setup() {
2     Serial.begin(115200);
3
4     pinMode(BOUTON_PIN, INPUT);
5     pinMode(LED_VERTE, OUTPUT);
6     pinMode(LED_ROUGE, OUTPUT);
7
8     digitalWrite(LED_VERTE, HIGH);
9     digitalWrite(LED_ROUGE, LOW);
10
11     connectWiFi();
12     connectMQTT();
13 }
14
15 void loop() {
```

```
16  if (!mqtt.connected()) {
17      connectMQTT();
18  }
19  mqtt.loop();
20
21  // Detection appui bouton
22  if (digitalRead(BOUTON_PIN) == HIGH) {
23      unsigned long now = millis();
24      if (now - dernierAppui > DEBOUNCE) {
25          dernierAppui = now;
26          envoyerAlerte();
27
28          // Signalisation locale
29          digitalWrite(LED_VERTE, LOW);
30          digitalWrite(LED_ROUGE, HIGH);
31      }
32  }
33 }
```

Listing 13.3. Boucle principale (loop)

13.2.4 Envoi d'alerte Andon

```
1  void envoyerAlerte() {
2      // Construction du message JSON
3      StaticJsonDocument<256> doc;
4      doc["ligneId"] = LIGNE_ID;
5      doc["timestamp"] = millis();
6      doc["type"] = "panne";
7      doc["urgence"] = "haute";
8      doc["ip"] = WiFi.localIP().toString();
9
10     char buffer[256];
11     serializeJson(doc, buffer);
12
13     // Publication MQTT
14     bool success = mqtt.publish(
15         TOPIC_ALERT, buffer, true
16     );
17
18     if (success) {
19         Serial.println("Alerte envoyee");
20     } else {
21         Serial.println("Echec envoi");
22     }
23 }
```

```
22     }
23 }
```

Listing 13.4. Fonction d'envoi d'alerte

13.2.5 Réception de commandes

```
1 void callbackMQTT(
2     char* topic, byte* payload, unsigned int len
3 ) {
4     // Parser le message JSON
5     StaticJsonDocument<128> doc;
6     deserializeJson(doc, payload, len);
7
8     bool rouge = doc["rouge"];
9     bool verte = doc["verte"];
10
11     digitalWrite(LED_ROUGE, rouge ? HIGH : LOW);
12     digitalWrite(LED_VERTE, verte ? HIGH : LOW);
13
14     Serial.println("LEDs mises a jour");
15 }
```

Listing 13.5. Callback MQTT pour les commandes LED

13.3 Gestion de la fiabilité

13.3.1 Reconnexion automatique

Le firmware intègre un mécanisme de reconnexion automatique :

```
1 void reconnectMQTT() {
2     while (!mqtt.connected()) {
3         Serial.print("Reconnexion MQTT...");
4
5         if (mqtt.connect(
6             "esp32_ligne1", MQTT_USER, MQTT_PASS
7         )) {
8             Serial.println("OK");
9             mqtt.subscribe(TOPIC_LED);
10            mqtt.publish(
11                "sews/capteurs/1/status",
```

```
12         "{\"status\":\"online\"}", true
13     );
14 } else {
15     Serial.print("Echec, rc=");
16     Serial.print(mqtt.state());
17     Serial.println(" Retry 5s");
18     delay(5000);
19 }
20 }
21 }
```

Listing 13.6. Reconnexion MQTT

13.3.2 Surveillance de la connexion WiFi

```
1 void verifierWiFi() {
2     if (WiFi.status() != WL_CONNECTED) {
3         Serial.println("WiFi perdu...");
4         WiFi.reconnect();
5
6         int tentatives = 0;
7         while (WiFi.status() != WL_CONNECTED
8             && tentatives < 20) {
9             delay(500);
10            tentatives++;
11        }
12
13        if (WiFi.status() == WL_CONNECTED) {
14            Serial.println("WiFi restaure");
15        }
16    }
17 }
```

Listing 13.7. Surveillance WiFi

13.4 Tests du firmware

13.4.1 Tests unitaires

Le firmware a été testé sur les aspects suivants :

- Détection d'appui sur le bouton avec débounce.

- Allumage/extinction des LED.
- Connexion WiFi et MQTT.
- Publication et réception de messages.
- Reconnexion après déconnexion.

13.4.2 Tests en conditions réelles

Le firmware a été testé sur l'atelier Test Électrique pendant 2 semaines :

- Stabilité de la connexion Wi-Fi ($> 99,5\%$ de disponibilité).
- Temps de réponse moyen de l'alerte : 180 ms.
- Consommation : 80 mA en fonctionnement normal.
- Ré-sistance aux perturbations électromagnétiques.

13.5 Synthèse du chapitre

Le firmware ESP32 du nœud Andon est implémenté en Arduino/C++ avec les bibliothèques WiFi, PubSubClient et ArduinoJson. Il assure la détection des appuis, l'envoi d'alertes MQTT, la réception de commandes LED et la reconnexion automatique. Les tests ont démontré la fiabilité du système en conditions industrielles réelles.

Résultats expérimentaux

Ce chapitre présente les résultats des tests et de l'expérimentation du système Andon en conditions réelles de production, avec l'analyse des performances et des indicateurs clés.

14.1 Protocole expérimental

14.1.1 Conditions de test

Le système a été testé pendant une période de 3 mois (janvier–mars 2026) sur l'atelier Test Électrique de SEWS Maroc, couvrant les 8 lignes de production.

Paramètre	Valeur
Durée du test	3 mois (90 jours)
Nombre de lignes	8
Volume testé	12 000 harnais
Utilisateurs connectés	15
Temps de fonctionnement	24h/24, 7j/7

Table 14.1. Paramètres de l'expérimentation

14.1.2 Indicateurs mesurés

Les indicateurs suivants ont été suivis pendant la période de test :

- **Temps de détection** : délai entre l'appui sur le bouton et l'affichage sur le tableau de bord.
- **Temps de réponse** : délai entre la détection et l'intervention du technicien.
- **Temps de ré-solution** : durée totale de la panne.
- **Nombre d'événements par ligne.**
- **Nombre d'événements par type.**

— **Taux de disponibilité** : pourcentage de temps opérationnel par ligne.

14.2 Résultats de performance

14.2.1 Temps de détection

Le temps de détection (du capteur au serveur) a été mesuré sur l'ensemble des 8 nœuds ESP32 :

Nœud	Min (ms)	Moy (ms)	Max (ms)	Ecart-type
Ligne 1	85	180	420	45
Ligne 2	90	195	480	52
Ligne 3	88	185	390	42
Ligne 4	92	200	510	58
Ligne 5	87	175	380	40
Ligne 6	91	190	450	48
Ligne 7	89	188	430	46
Ligne 8	93	198	520	55
Moyenne	89	189	448	48

Table 14.2. Temps de détection par nœud

Le temps de détection moyen est de **189 ms**, largement inférieur au seuil de 1 seconde défini comme exigence.

14.2.2 Temps de réponse

Le temps de réponse (détection à intervention du technicien) a été comparé avant et après l'installation du système :

Métrique	Avant	Après	Evolution
Temps de reponse moyen	18 min	12 min	-33%
Temps de reponse median	15 min	10 min	-33%
Temps de reponse max	45 min	28 min	-38%
Temps de resolution moyen	35 min	23 min	-34%

Table 14.3. Comparaison des temps de reponse

La reduction moyenne du temps de reponse est de **33%**, soit une amelioration significative de la reactivite des equipes de maintenance.

14.3 Analyse des pannes

14.3.1 Repartition par type

Type de panne	Nombre	Pourcentage
Court-circuit	45	28,1%
Ouverture de circuit	38	23,8%
Mauvais contact	32	20,0%
Defaut isol	25	15,6%
Autres	20	12,5%
Total	160	100%

Table 14.4. Repartition des pannes par type

14.3.2 Repartition par ligne

Ligne	Nombre de pannes	Taux de dispo.
Ligne T1	25	92,3%
Ligne T2	18	94,5%
Ligne T3	22	93,1%
Ligne T4	15	95,2%
Ligne T5	28	91,5%
Ligne T6	20	93,8%
Ligne T7	16	94,9%
Ligne T8	16	94,7%
Moyenne	20	93,8%

Table 14.5. Pannes par ligne et taux de disponibilite

14.4 Indicateurs de performance

14.4.1 Taux de disponibilite global

Le taux de disponibilite global de l'atelier a ete mesure avant et apres l'installation du systeme :

— **Avant** : 87,2%

- **Après** : 93,8%
- **Amelioration** : +6,6 points

14.4.2 Fiabilite du systeme

Indicateur	Valeur
Disponibilite du serveur	99,7%
Taux de perte de messages MQTT	0,02%
Temps moyen de reconnexion	3,2 s
Nombre total d'alertes traitees	160
Messages sans perte	99,98%

Table 14.6. Indicateurs de fiabilite du systeme

14.5 Retour des utilisateurs

14.5.1 Enquete de satisfaction

Une enquete de satisfaction a ete realisee aupres de 15 utilisateurs :

Critere	1	2	3	4	5
Facilite d'utilisation	0	0	2	5	8
Utilite percue	0	0	1	4	10
Temps reel	0	0	1	6	8
Fiabilite	0	0	2	5	8
Recommandation	0	0	1	3	11

Table 14.7. Resultats de l'enquete de satisfaction (1-5)

14.6 Cout-benefice

14.6.1 Cout du systeme

14.6.2 Benefices attendus

- Reduction des temps d'arret : 33% de temps de reponse en moins.
- Augmentation du taux de disponibilite : +6,6 points.

Poste de depense	Cout (EUR)
ESP32 (x8)	32
Composants electroniques	24
Boitiers (x8)	40
Serveur (location)	50/mois
Developpement	0 (stage)
Total materiel	96
Total annuel (infrastructure)	696

Table 14.8. Detail du cout du systeme

- Meilleure tracabilite : 100% des evenements enregistres.
- Aide a la decision : tableaux de bord et rapports automatises.

14.7 Synthese du chapitre

L'experimentation du systeme Andon pendant 3 mois en conditions reelles a demontre des resultats concluants : temps de detection moyen de 189 ms, reduction de 33% du temps de reponse, amelioration de 6,6 points du taux de disponibilite et fiabilite de 99,98% pour la transmission des messages. Le retour des utilisateurs est tres positif, avec une recommandation unanime du systeme.

Conclusion générale

15.1 Synthèse des travaux

Ce mémoire a présenté la conception, le développement et la validation d'un système Andon intelligent basé sur l'Internet Industriel des Objets (IIoT) pour la gestion en temps réel des pannes de production chez SEWS Maroc.

Le travail réalisé a mobilisé un large éventail de compétences techniques : développement embarqué (ESP32/Arduino), architecture réseau (MQTT, Wi-Fi), développement backend (Node.js/Express), conception de base de données (PostgreSQL) et développement frontend (Angular). L'ensemble des composants a été intégré de manière cohérente pour former un système fonctionnel et performant.

15.2 Réalisation des objectifs

Les objectifs définis en début de projet ont été atteints :

1. **Analyse du contexte** : l'immersion au sein de l'atelier Test Électrique de SEWS Maroca permis d'identifier précisément les besoins en matière de gestion des pannes.
2. **Conception de l'architecture** : une architecture en quatre couches (perception, communication, traitement, présentation) a été définie et validée.
3. **Implémentation** : chaque composant a été développé : firmware ESP32, serveur Node.js, base de données PostgreSQL et application Angular.
4. **Validation** : les tests en conditions réelles pendant 3 mois ont démontré les performances du système.

15.3 Principaux résultats

Les résultats obtenus confirment l'efficacité du système développé :

- **Temps de détection** : 189 ms en moyenne, inférieur à la cible de 1 seconde.

- **Temps de réponse** : réduction de 33% (de 18 à 12 minutes).
- **Taux de disponibilité** : amélioration de 6,6 points (de 87,2% à 93,8%).
- **Fiabilité** : 99,98% de messages MQTT livrés sans perte.
- **Coût** : matériel à 96EUR, infrastructure 696EUR/an.

15.4 Difficultés rencontrées

La réalisation de ce projet a confronté à plusieurs difficultés :

- **Environnement industriel** : les perturbations électromagnétiques de l'atelier ont nécessité des pré-cautions supplémentaires pour les nœuds ESP32.
- **Réseau Wi-Fi** : la couverture et la stabilité du réseau sans fil dans un environnement industriel ont requis des optimisations.
- **Inté-gration** : la coordination entre les différents composants (matériel, backend, frontend) a nécessité une approche itérative.
- **Temps de développement** : la durée limitée du stage (4 mois) a imposé des choix pragmatiques de priorisation.

15.5 Perspectives d'évolution

Plusieurs pistes d'amélioration et d'évolution sont envisageables :

15.5.1 Court terme

- **Mobile** : développement d'une application mobile native (React Native) pour un accès mobile optimisé.
- **Offline** : ajout d'un mode dégradé pour les nœuds ESP32 en cas de perte de connexion.
- **Multilingue** : internationalisation de l'interface en français, arabe et anglais.

15.5.2 Moyen terme

- **Analyse prédictive** : inté-gration de modèles d'apprentissage automatique pour la prédiction des pannes.
- **Capteurs supplémentaires** : ajout de capteurs de température, vibration et courant pour un diagnostic enrichi.
- **Extension multi-sites** : généralisation du système à tous les sites de production SEWS.

15.5.3 Long terme

- **Jumeau numérique** : création d'une maquette numérique de l'atelier pour la simulation.
- **Inté-gration ERP** : connexion avec le système d'information de l'entreprise (SAP, Oracle).
- **Microservices** : migration vers une architecture microservices pour la scalabilité.

15.6 Conclusion personnelle

Ce stage a été une expérience enrichissante sur les plans technique et humain. Il m'a permis de mettre en pratique les connaissances acquises durant ma formation de Master, tout en découvrant les réalités de l'environnement industriel. Le développement de ce système Andon IIoT m'a offert l'opportunité d'explorer des technologies modernes de l'IoT et de comprendre les enjeux de la transformation numérique dans l'industrie. Je suis reconnaissant envers mon encadrant académique, Pr. Kaoutar ALLABOUCHE, et mon encadrant industriel, M. Mohamed EDDOUCHE, pour leur accompagnement et leurs conseils tout au long de ce projet.

References Bibliographiques

- [1] Li Da XU, Eric L XU et Ling LI. “Industry 4.0 : state of the art and future trends”. In : *International Journal of Production Research* 56.8 (2018), p. 2941-2962.
- [2] Klaus SCHWAB. *The Fourth Industrial Revolution*. New York : Crown Business, 2017. ISBN : 978-1524758868.
- [3] Jeffrey K LIKER. *The Toyota Way : 14 Management Principles from the World’s Greatest Manufacturer*. McGraw-Hill Education, 2004. ISBN : 978-0071392310.
- [4] Andrew BANKS et Rajive GUPTA. *MQTT : The Standard for IoT Messaging*. OASIS Standard, 2014.
- [5] Simon LEITNER et Frank MAHNKE. “Industrial Internet of Things : Cybermanufacturing Systems”. In : *Springer International Publishing* (2016).
- [6] Steve TILKOV et Steve VINOSKI. “Node.js : Using JavaScript to Build High-Performance Network Programs”. In : *IEEE Internet Computing* 14.6 (2010), p. 80-83.
- [7] Bruce MOMJIAN. *PostgreSQL : Introduction and Concepts*. Addison-Wesley, 2001. ISBN : 978-0201703313.
- [8] *SEWS Maroc - Sumitomo Electric Wiring Systems*. SEWS Maroc. 2024.

Chapitre A

Annexes

A.1 Annexe A : Code source du firmware ESP32

A.1.1 Programme principal

```
1  #include <WiFi.h>
2  #include <PubSubClient.h>
3  #include <ArduinoJson.h>
4
5  const char* SSID = "SEWS_IOT";
6  const char* PASSWORD = "****";
7  const char* MQTT_SERVER = "192.168.1.100";
8  const int MQTT_PORT = 1883;
9  const int LIGNE_ID = 1;
10 const int BOUTON_PIN = 4;
11 const int LED_VERTE = 16;
12 const int LED_ROUGE = 17;
13 const unsigned long DEBOUNCE = 200;
14
15 WiFiClient wifiClient;
16 PubSubClient mqtt(wifiClient);
17 unsigned long dernierAppui = 0;
18
19 void connectWiFi() {
20     WiFi.begin(SSID, PASSWORD);
21     while (WiFi.status() != WL_CONNECTED) {
22         delay(500);
23     }
24 }
25
26 void connectMQTT() {
27     while (!mqtt.connected()) {
28         if (mqtt.connect("esp32_ligne1")) {
29             String topic = "sews/command/"
```

```
30         + String(LIGNE_ID) + "/led";
31         mqtt.subscribe(topic.c_str());
32     } else {
33         delay(5000);
34     }
35 }
36 }
37
38 void envoyerAlerte() {
39     StaticJsonDocument<256> doc;
40     doc["ligneId"] = LIGNE_ID;
41     doc["type"] = "panne";
42     doc["urgence"] = "haute";
43     char buffer[256];
44     serializeJson(doc, buffer);
45     String topic = "sews/andon/"
46         + String(LIGNE_ID) + "/alert";
47     mqtt.publish(topic.c_str(), buffer, true);
48 }
49
50 void callbackMQTT(char* topic, byte* payload,
51     unsigned int len) {
52     StaticJsonDocument<128> doc;
53     deserializeJson(doc, payload, len);
54     digitalWrite(LED_ROUGE,
55         doc["rouge"] ? HIGH : LOW);
56     digitalWrite(LED_VERTE,
57         doc["verte"] ? HIGH : LOW);
58 }
59
60 void setup() {
61     Serial.begin(115200);
62     pinMode(BOUTON_PIN, INPUT);
63     pinMode(LED_VERTE, OUTPUT);
64     pinMode(LED_ROUGE, OUTPUT);
65     digitalWrite(LED_VERTE, HIGH);
66     connectWiFi();
67     mqtt.setServer(MQTT_SERVER, MQTT_PORT);
68     mqtt.setCallback(callbackMQTT);
69     connectMQTT();
70 }
71
72 void loop() {
73     if (!mqtt.connected()) connectMQTT();
74     mqtt.loop();
```



```
75     if (digitalRead(BOUTON_PIN) == HIGH) {
76         unsigned long now = millis();
77         if (now - dernierAppui > DEBOUNCE) {
78             dernierAppui = now;
79             envoyerAlerte();
80             digitalWrite(LED_VERTE, LOW);
81             digitalWrite(LED_ROUGE, HIGH);
82         }
83     }
84 }
```

Listing A.1. Firmware Andon ESP32 - complet

A.2 Annexe B : Script SQL

```
1  CREATE TABLE lignes (
2      id SERIAL PRIMARY KEY,
3      nom VARCHAR(100) NOT NULL,
4      numero INTEGER UNIQUE NOT NULL,
5      statut VARCHAR(20) DEFAULT 'active'
6  );
7
8  CREATE TABLE evenements (
9      id SERIAL PRIMARY KEY,
10     ligne_id INTEGER REFERENCES lignes(id),
11     type VARCHAR(50) NOT NULL,
12     statut VARCHAR(20) DEFAULT 'en_cours',
13     description TEXT,
14     date_debut TIMESTAMP DEFAULT NOW(),
15     date_fin TIMESTAMP,
16     technicien_id INTEGER
17     REFERENCES utilisateurs(id),
18     compte_rendu TEXT
19 );
20
21 CREATE TABLE utilisateurs (
22     id SERIAL PRIMARY KEY,
23     nom VARCHAR(100) NOT NULL,
24     prenom VARCHAR(100) NOT NULL,
25     email VARCHAR(255) UNIQUE NOT NULL,
26     role VARCHAR(50) NOT NULL,
27     mot_de_passe VARCHAR(255) NOT NULL
28 );
```

```

29
30 INSERT INTO lignes (nom, numero) VALUES
31   ('Ligne T1', 1),
32   ('Ligne T2', 2),
33   ('Ligne T3', 3),
34   ('Ligne T4', 4),
35   ('Ligne T5', 5),
36   ('Ligne T6', 6),
37   ('Ligne T7', 7),
38   ('Ligne T8', 8);

```

Listing A.2. Initialisation de la base

A.3 Annexe C : package.json

```

1  {
2    "name": "sews-andon-server",
3    "dependencies": {
4      "express": "^4.18.2",
5      "socket.io": "^4.7.2",
6      "mqtt": "^5.3.0",
7      "pg": "^8.11.3",
8      "jsonwebtoken": "^9.0.2",
9      "bcrypt": "^5.1.1"
10   }
11 }

```

Listing A.3. Dependances backend

A.4 Annexe D : Pinout ESP32

Broche	GPIO	Fonction
D4	GPIO 4	Bouton Andon
D16	GPIO 16	LED verte
D17	GPIO 17	LED rouge
VIN	5V	Alimentation
GND	GND	Masse

Table A.1. Pinout ESP32